

SQLP 과목 III 튜닝 스타일 모의문제 · 6회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

블록 I/O를 효율화하는 방향과 그 수단(Multiblock I/O·Prefetch·Sequential 액세스)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 넓은 범위를 훑는 Full Table Scan은 물리적으로 인접한 블록을 저장 순서대로 읽는 Sequential 액세스이므로, 인접 블록을 한 번의 I/O Call에 묶는 Multiblock I/O와 잘 맞아 같은 블록 수를 읽어도 I/O Call 횟수가 줄어든다.
- ② I/O 효율화의 목표는 물리적 I/O를 늘려서라도 논리적 I/O를 줄이는 데 있으며, 버퍼 캐시 적중률이 높아질수록 디스크를 직접 때리는 물리적 블록 I/O가 오히려 증가한다.
- ③ Prefetch는 앞으로 필요할 것으로 예상되는 블록의 읽기를 미리 요청해 두는 기법으로, 개별 블록을 하나씩 기다리는 순차적 대기를 겹치게 만들어 I/O 지연을 감추는 효과를 낸다.
- ④ 인덱스 ROWID로 테이블의 한 블록을 임의 위치에서 집어 오는 Random 액세스는 흩어진 블록을 건별로 읽어야 해 Single Block I/O로 처리되는 경향이 있고, 이는 연속 블록을 묶어 읽는 Sequential 액세스와 대비된다.

2

Redo·Undo의 역할과 Write-Ahead Logging(로그 선기록) 규칙, 그리고 커밋 시점의 동작에 대한 설명으로 가장 적절한 것은?

- ① 커밋을 내리면 그 트랜잭션이 변경한 데이터 블록(더티 버퍼)을 DBWR가 데이터파일에 먼저 내려쓴 뒤에야 LGWR가 Redo를 로그 파일에 기록하며 커밋이 완료되므로, 커밋 응답 시간은 더티 블록 수에 비례해 길어진다.
- ② Write-Ahead Logging 규칙에 따라 변경된 데이터 블록을 데이터파일에 반영하기 전에 그 변경을 담은 Redo가 로그 파일에 먼저 기록되어야 하며, 커밋 시점에는 LGWR가 해당 트랜잭션의 Redo를 로그 파일에 내려써 변경의 영속성을 확보한다.
- ③ Undo는 트랜잭션 롤백에만 쓰이는 정보라, 다른 세션이 과거 시점의 이미지를 재구성해 읽는 읽기 일관성은 Undo가 아니라 로그 파일의 Redo를 거꾸로 훑어 만들어지며, Undo 세그먼트는 커밋과 동시에 해제돼 재사용 대상이 된다.
- ④ Redo는 장애 복구 시 아직 커밋되지 않은 변경을 취소(rollback)하는 데 쓰이는 정보이고, Undo는 이미 커밋된 변경을 다시 적용(roll forward)해 살려 내는 데 쓰인다. Write-Ahead Logging도 이 배분을 따라 Undo를 먼저 기록하라는 규칙이다.

3

현렬 관리 시스템의 야간 배치가 현렬기록 테이블에서 당일 현렬 6만 건을 조회하면서, 각 건마다 사용자 정의 함수 f_현액판정(...)을 SELECT 목록에서 호출한다. 이 함수는 내부에 참조 테이블을 찾는 SELECT 한 건을 담고 있어 행마다 한 번씩 수행된다. 아래 TKPROF는 이 배치 세션의 [비재귀 statements 합계]와 [재귀 statements 합계]를 나눠 집계한 결과이다. 애플리케이션이 만든 User Call(네트워크 왕복)과 서버 내부 Recursive Call을 구분할 때, 다음 중 옳은 진단은?

아래

OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS

Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	1	0.010	0.012	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	301	1.500	3.500	500	5000	0	60000
Total	303	1.510	3.512	500	5000	0	60000

OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS

Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	1	0.001	0.001	0	0	0	0
Execute	60000	6.000	6.500	0	0	0	0
Fetch	60000	12.000	40.000	4000	480000	0	60000
Total	120001	18.001	46.501	4000	480000	0	60000

- ① 재귀 Fetch 60,000회와 재귀 Execute 60,000회는 결국 클라이언트를 12만 번 오간 왕복이고, 비재귀 Fetch 301회는 그중 일부만 잡힌 것이다. 배열 인출 크기를 지금의 200행에서 2,000행으로 키우면 이 12만 회 왕복이 1/10로 줄어, 재귀 Elapsed 46.501초가 함께 해소되고 응답이 3.512초 수준으로 내려간다.
- ② 세션의 DB Call은 비재귀 303회와 재귀 120,001회를 합한 120,304회이고 이것이 전부 애플리케이션이 만든 User Call이므로, 6만 행을 처리하려고 왕복을 12만 번 한 셈이다. 재귀 Fetch 60,000회가 그중 절반이니 커넥션 풀링과 세션 재사용으로 왕복 수를 줄이면 46.501초의 대부분을 되돌릴 수 있다.
- ③ 비재귀 Elapsed는 3.512초뿐이고 재귀 46.501초의 정체는 재귀 Fetch가 쓴 Query 480,000 블록에 있다. 이는 비재귀 Query 5,000의 96배이므로, 버퍼 캐시를 키워 논리 읽기 480,000을 줄이면 재귀 CPU 18.001초와 나머지 대기가 함께 사라져 배치가 정상화된다.
- ④ 애플리케이션이 만든 User Call(왕복)은 비재귀 블록의 303회뿐이고, 재귀 120,001회는 함수가 행마다 서버 안에서 만든 Recursive Call이라 왕복을 늘리지 않는다. 46.5초 병목은 60,000회 재귀 수행이라는 별개의 축에 있어, 배열 크기 상향(303회 축)이 아니라 행별 재귀 SQL 제거로만 줄어든다.

4

AWR/ASH 보고서로 응답 시간을 분석할 때의 응답 시간 = CPU + Wait 구조와 DB Time, Top 대기 이벤트 해석에 대한 설명으로 가장 적절한 것은?

- ① 응답 시간은 Service Time과 CPU Time, 그리고 Wait Time을 각각 따로 집계해 더한 값이므로, AWR의 Top Timed Events에서 DB CPU 행과 서비스 시간 항목을 별개의 두 항목으로 놓고 대기 이벤트 시간까지 세 항목을 합산해야 정확한 응답 시간이 나온다.
- ② 응답 시간은 CPU를 실제로 소비한 Service Time과 자원을 확보하려 기다린 Wait Time으로 나뉘며, AWR의 Top Timed Events에서 상위 이벤트가 DB Time에서 차지하는 비중을 보면 병목이 CPU 쪽인지 특정 대기 쪽인지 가늠할 수 있다.
- ③ AWR 보고서의 DB Time은 두 스냅샷 사이의 실제 경과 시간(wall-clock elapsed)과 같은 값이므로, DB Time이 경과 시간의 몇 배로 찍혔다면 스냅샷 구간이 잘못 잡혔다는 뜻이고, Top Timed Events의 비중도 DB Time이 아니라 경과 시간으로 나눠 읽어야 한다.
- ④ 대기 이벤트 목록에서 db file sequential read처럼 이름에 대기가 드러난 이벤트의 시간만 Wait Time이자 응답 시간이고, CPU를 쓴 시간은 일한 시간이라 응답 시간에서 빼야 하므로, Top Timed Events의 DB CPU 행은 병목 판단에서 제외하는 것이 옳다.

게임 운영팀이 게임접속로그 테이블(약 1억 2천만 건)에서 일자·플랫폼별 접속 수를 집계·정렬해 리포트로 내리는 SQL의 TKPROF 결과이다. 결과는 3,000행뿐인데 응답이 55초에 달한다. 아래 트레이스에 대한 해석으로 가장 적절한 것은?

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.005	0.005	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	301	6.995	54.995	300000	2000000	0	3000
Total	303	7.000	55.000	300000	2000000	0	3000
Rows	Row Source Operation						
3000	SORT ORDER BY (cr=2000000 pr=300000 pw=0 time=54900000 us)						
3000	HASH GROUP BY (cr=2000000 pr=300000 pw=0 time=54850000 us)						
120000000	TABLE ACCESS FULL 게임접속로그 (cr=2000000 pr=300000 pw=0 time=53000000 us)						

- ① CPU Time 7초가 Elapsed 55초를 끌고 가는 CPU 바운드가. Full Scan이 훑은 1억 2천만 행을 3,000행으로 접는 HASH GROUP BY의 해시 연산이 전부 CPU 작업이고 pw=0이라 디스크로 새는 부분도 없기 때문이므로, 병렬도(DOP)를 4로 올려 CPU를 여러 프로세스에 분산하는 것이 최우선 처방이다.
- ② 결과 3,000행을 만드는 SORT ORDER BY가 time=54.9초를 안고 있으므로 병목은 정렬이다. 일자·플랫폼 정렬 컬럼에 인덱스를 만들어 SORT ORDER BY를 없애면 이 54.9초가 사라지고, 그 아래 HASH GROUP BY도 인덱스 순서를 물려받아 SORT GROUP BY로 바뀌며 가벼워진다.
- ③ 논리 I/O 200만 블록 중 30만(15%)을 디스크에서 읽어 BCHR은 약 85%이고, CPU 7초는 Elapsed 55초의 약 12.7%뿐이라 나머지 약 48초가 대기다. cr·pr·time이 모두 게임접속로그 Full Scan(cr=2,000,000, pr=300,000, time=53초)에 몰려 있으므로 병목은 1억 2천만 건 Full Scan I/O다.
- ④ pr 300,000과 BCHR 85%는 정렬이 PGA를 넘겨 디스크로 스푼했다는 신호다. SORT ORDER BY의 time=54.9초 가운데 대부분이 이 스푼 대기이므로, PGA를 키워 정렬을 메모리 안에서 끝내면 Elapsed 55초에서 CPU 7초를 뺀 약 48초의 대기가 사라져 응답이 7초대가 된다.

전기차 충전 관제 시스템의 충전세션 테이블(약 870만 건)에는 결합 인덱스 충전_IX = (충전소코드, 충전시작일시)가 있다. 충전소명을 함께 얻기 위해 충전소 테이블(기본키 충전소_PK = 충전소코드)과 조인하는 다음 SQL의 실행계획이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아래

```
SELECT s.충전세션ID, s.충전시작일시, s.충전량, t.충전소명
FROM 충전세션 s, 충전소 t
WHERE s.충전소코드 = 'C15'
      AND s.충전시작일시 >= DATE '2026-06-01'
      AND s.충전시작일시 < DATE '2026-07-01'
      AND s.결제방식 = '앱'
      AND t.충전소코드 = s.충전소코드;
```

Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(312	2700	135000)
1	0	NESTED LOOPS	(312	2700	135000)
2	1	TABLE ACCESS BY INDEX ROWID 충전세션	(307	2700	94500)
3	2	INDEX RANGE SCAN 충전_IX	(22	9000	)
4	1	TABLE ACCESS BY INDEX ROWID 충전소	(1	1	15)
5	4	INDEX UNIQUE SCAN 충전소_PK	(0	1	)

- ① 충전소코드(등치)와 충전시작일시(범위)까지가 충전_IX의 인덱스 액세스 조건으로 INDEX RANGE SCAN에서 처리(Card 9,000)되고, 인덱스에 없는 결제방식은 TABLE ACCESS BY INDEX ROWID 단계 필터로 걸려져 Card가 2,700으로 줄어든다.
- ② 결제방식='앱' 조건도 충전_IX의 인덱스 필터로 INDEX RANGE SCAN(Id 3) 단계에서 걸려져 인덱스 스캔 범위를 좁히는 데 사용된다. Id 3의 Cost 22가 세 조건을 인덱스에서 소화했다는 증거이며, Id 2의 Card 2,700은 그 결과를 물려받은 값이다.
- ③ 선두 칼럼 뒤 충전시작일시가 범위(>=, <) 조건이므로, 선두 칼럼 충전소코드의 등치도 인덱스 액세스 조건이 되지 못하고 스캔 후 필터로만 처리된다. Id 3이 낸 Card 9,000도 870만 건짜리 인덱스를 통째로 훑은 뒤 필터로 걸러 남긴 결과다.
- ④ INDEX RANGE SCAN(Id 3)이 반환한 9,000건이 곧 이 SQL의 최종 결과 건수이며, Id 2 TABLE ACCESS BY INDEX ROWID는 충전량·충전세션ID 값을 채우기만 할 뿐 건수 감소를 일으키지 않는다. Id 1의 Card 2,700은 충전소와 조인하며 짝을 못 찾은 건이 빠진 결과다.

7

온라인 경매 시스템에서 경매체결 테이블(약 4,000만 건, 블록당 약 100행 → 약 40만 블록)을 특정 물품카테고리 조건으로 조회한다. 카테고리 컬럼의 인덱스 경매체결_N1로 20만 건을 뽑는데, 인덱스를 타는데도 25초가 걸린다. 아래 TKPROF에서 지연의 원인을 직접 지목한 것으로 옳은 것은?

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.004	0.004	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	401	3.000	24.996	40000	191400	0	200000
Total	403	3.004	25.000	40000	191400	0	200000
Rows	Row Source Operation						
200000	TABLE ACCESS BY INDEX ROWID 경매체결 (cr=191400 pr=40000 pw=0 time=24500000 us)						
200000	INDEX RANGE SCAN 경매체결_N1 (cr=1400 pr=200 pw=0 time=350000 us)						

- ① 테이블 액세스 자체 블록 I/O 190,000이 액세스 ROWID 수 200,000과 거의 같아 클러스터링 팩터가 최악이다. 이 190,000이 전체 cr의 99.3%와 time 24.5초를 차지하고 40만 블록 테이블의 47.5%를 단일 블록 랜덤 읽기로 훑어 손익분기점을 넘었으므로, 커버링 인덱스로 테이블 액세스를 제거하는 것이 답이다.
- ② 병목은 INDEX RANGE SCAN이다. 이 인덱스가 20만 건을 걸러내며 cr 1,400·pr 200을 쓰는데, 20만 개의 ROWID를 뽑느라 리프 블록이 조각나 time=0.35초가 붙은 것이다. 인덱스를 리빌드해 리프를 조밀하게 만들면 ROWID가 정렬돼 상위 테이블 액세스의 24.5초까지 함께 정리되므로 25초 응답이 1초대로 줄어든다.
- ③ Fetch 401회는 20만 행을 클라이언트로 실어 나른 왕복이고, 왕복 한 번에 24.5초/401 ≈ 61밀리초의 지연이 붙어 24.5초가 쌓였다. 배열 인출 크기를 지금의 500행에서 5,000행으로 키워 Fetch를 41회로 줄이면 왕복 지연이 1/10이 되어 테이블 액세스 지연도 함께 사라진다.
- ④ 물리 읽기 Disk 40,000이 time=24.5초의 근본 원인이다. 40,000블록을 단일 블록으로 읽으며 회당 약 0.6밀리초씩 24초가 쌓인 것이므로, 버퍼 캐시를 키워 이 40,000을 논리 읽기로 돌리면 cr 191,400만 남아 응답이 최적화되고 인덱스 구성은 손댈 필요가 없다.

8

B-Tree 인덱스의 구조(루트·브랜치·리프)와 수직 탐색, 그리고 Index Unique/Range Scan에 대한 설명으로 가장 적절하지 않은 것은?

- ① B-Tree 인덱스는 루트 블록에서 브랜치를 거쳐 리프까지 위에서 아래로 수직 탐색하며, 시작 리프에 닿기까지 인덱스 높이(BLEVEL+1)에 해당하는 블록을 읽는다.
- ② 리프 블록끼리는 좌우 양방향 포인터로 이어져 있어, 시작 리프에 도달한 뒤에는 수평으로 이웃 리프를 따라가며 조건을 만족하는 엔트리를 이어 읽을 수 있다.
- ③ Index Unique Scan은 유일성이 보장된 키에 등가 조건을 주었을 때 최대 한 건을 찾으면 즉시 멈추므로, 리프를 수평으로 이어 읽는 과정이 없다.
- ④ 인덱스 높이가 낮을수록 리프 하나에 담기는 엔트리 수가 늘어 Range Scan이 훑는 리프 블록 수도 함께 줄어들므로, 인덱스 높이가 Range Scan이 읽는 리프 블록 수를 결정하는 주된 요인이다.

9

결합 인덱스에서 액세스 조건과 필터 조건의 구분, 그리고 그것이 인덱스 스캔 효율(읽는 리프 블록 수)에 미치는 영향에 대한 설명으로 가장 적절하지 않은 것은?

- ① 결합 인덱스의 구성 컬럼에 걸린 조건이라면 그것이 액세스 조건이든 필터 조건이든 인덱스 스캔이 훑는 리프 구간(스캔 범위)을 좁혀 주므로, 필터 조건을 많이 걸수록 읽어야 할 리프 블록 수가 줄어든다.
- ② 액세스 조건은 인덱스 스캔의 시작점과 종료점을 결정해 훑을 리프 구간 자체를 줄이므로, 읽어야 할 리프 블록 수를 직접 감소시킨다.
- ③ 필터 조건은 스캔 구간 안의 각 엔트리를 하나씩 평가해 걸러 낼 뿐 스캔 구간의 크기를 줄이지 못하므로, 결국 버려질 엔트리가 놓인 리프 블록까지 읽는 비용이 든다.
- ④ 결합 인덱스에서는 선두 컬럼부터 등치로 이어지다 처음 만난 범위 조건까지가 액세스 조건이 될 수 있고, 그 뒤 컬럼의 조건은 인덱스에 값이 있어도 필터 조건으로만 작동한다.

10

부동산 실거래 분석 배치에서 단지마스터 테이블(약 5만 건)과 실거래내역 테이블(약 7,500만 건)을 단지코드로 조인해 지역별 평균 거래가를 집계한다. 단지마스터에는 단지코드·지역·단지명이, 실거래내역에는 최근 6개월 거래만 걸러 읽는 조건이 있다. 다음 실행계획에 대한 설명으로 가장 적절한 것은?

아래

Id	Operation	Name
0	SELECT STATEMENT	
1	SORT ORDER BY	
2	HASH GROUP BY	
* 3	HASH JOIN	
4	TABLE ACCESS FULL	단지마스터
* 5	TABLE ACCESS FULL	실거래내역

- ① Id 5 실거래내역(7,500만 건)이 HASH JOIN의 Build Input이다. 옵티마이저는 두 입력 중 큰 쪽을 Build로 삼아 해시 테이블에 담아 두고, 작은 Id 4 단지마스터(5만 건)를 Probe로 훑어보내 Probe 반복 횟수를 5만 회로 줄이려 한 것이다.
- ② 이 조인의 병목은 Id 4 단지마스터 5만 건을 Hash Area에 올려 해시 테이블을 짓는 Build 단계에 있다. 그 위의 Id 2 HASH GROUP BY와 Id 1 SORT ORDER BY가 같은 작업 영역을 나눠 쓰기 때문이며, Probe 쪽 Id 5는 인덱스가 없어서 가벼운 작업이다.
- ③ HASH JOIN(Id 3)은 Id 4 단지마스터의 한 행마다 Id 5 실거래내역 전체를 반복 스캔해 단지코드를 매칭하므로, 5만 번의 Full Scan이 겹쳐 일어난다. 실거래내역의 단지코드에 조인 인덱스가 없어 그 반복 탐색이 TABLE ACCESS FULL로 떨어진 것이 느린 이유다.
- ④ Id 4 단지마스터가 Build Input(작은 쪽)으로 먼저 읽혀 Hash Area에 해시 테이블로 구축되고, Id 5 실거래내역이 Probe Input으로 한 번 스캔되며 해시 테이블을 조회해 매칭된다. 조인 순서는 Build → Probe이고 큰 테이블은 한 번만 훑는다.

11

소트 머지 조인(Sort Merge Join)의 처리 방식과, NL 조인·해시 조인 사이에서의 선택 기준에 대한 설명으로 가장 적절한 것은?

- ① 소트 머지 조인은 '정렬 후 병합'이라는 이름 그대로 한쪽 집합만 조인 키로 정렬해 두면 되고, 다른 쪽은 그 정렬 순서를 따라가며 탐색한다. 그래서 정렬 비용이 한 번만 들어 해시 조인의 Build 부담보다 가볍고, 정렬되지 않은 쪽이 아무리 커도 추가 정렬 없이 병합할 수 있다.
- ② 소트 머지 조인은 양쪽 집합을 각각 조인 키로 정렬한 뒤 정렬 순서를 따라 병합하므로 등치뿐 아니라 부등호·범위(BETWEEN) 같은 비등치 조인도 처리할 수 있고, 정렬 부담이 크지 않거나 이미 정렬된 입력을 얻을 수 있을 때 해시 조인의 대안이 된다.
- ③ 정렬 단계가 개입하므로 소트 머지 조인은 대량 데이터 조인에서 해시 조인보다 느릴 수밖에 없어, 소량 데이터 조인 전용으로 분류된다. 양쪽 정렬이 작업 영역을 넘기면 디스크로 스푼하는데, 해시 조인은 작은 쪽만 메모리에 올리므로 스푼 위험이 더 낮기 때문이다.
- ④ 소트 머지 조인은 양쪽을 정렬해 병합하는 방식이므로, 조인 키에 인덱스가 있어 이미 정렬된 순서로 읽을 수 있으면 정렬 단계가 사라져 사실상 NL 조인과 동일한 처리 방식이 된다. 이때는 드라이빙 집합의 각 행마다 인너 인덱스를 탐색하게 되므로 조인 순서가 성능을 좌우한다.

옵티마이저의 쿼리 변환 중 GROUP BY·DISTINCT가 포함된 복합 뷰의 병합(복합 뷰 Merging)에 대한 설명으로 가장 적절하지 않은 것은?

- ① 단순 뷰 Merging은 뷰 정의를 바깥 쿼리에 그대로 풀어 한 블록으로 합치는 반면, GROUP BY·DISTINCT가 든 복합 뷰의 병합은 집계·중복 제거 연산까지 함께 재배치해야 하므로 단순 병합과 처리 방식이 다르다.
- ② 복합 뷰 Merging에서 옵티마이저는 집계를 조인 뒤로 미뤄 수행하거나(조인 먼저) 뷰에서 먼저 집계한 뒤 조인하는 두 형태를 비용으로 비교해 고를 수 있다.
- ③ GROUP BY나 DISTINCT가 포함된 복합 뷰는 집계·중복 제거 결과를 먼저 확정해야 하므로 병합 대상에서 제외되며, 옵티마이저는 뷰를 개별 블록으로 수행한 결과를 바깥 쿼리와 조인할 뿐 그 수행 순서를 바꾸지는 못한다.
- ④ 복합 뷰 Merging이 언제나 유리한 것은 아니어서 옵티마이저는 비용으로 병합 여부를 판단하며, NO_MERGE 힌트로 병합을 막거나 MERGE 힌트로 병합을 유도할 수 있다.

야간 배치가 차량출입 테이블에서 차량구분별 건수를 반복 조회한다. 차량구분 코드는 여러 종류('CAR'·'BUS'·'TRK'·'BIKE'...)이고, 배치는 코드 배열을 순회하며 코드마다 한 번씩 카운트한다. 개발자는 아래 두 방식을 두고 고민 중이다. 방식 (가)는 DBMS_SQL로 바인드 플레이스홀더를 심어 루프 밖에서 1회 PARSE하고 루프 안에서는 값만 바꿔 EXECUTE를 반복하며, 방식 (나)는 EXECUTE IMMEDIATE(NDS)에 차량구분 값을 리터럴로 결합해 루프마다 문장을 새로 만든다. 동적 SQL과 커서 공유에 대한 설명으로 가장 적절하지 않은 것은?

아 래

```
-- 방식 (가) : DBMS_SQL - 루프 밖 1회 PARSE, 루프 안 BIND+EXECUTE 반복
c := DBMS_SQL.OPEN_CURSOR;
DBMS_SQL.PARSE(c, 'SELECT COUNT(*) FROM 차량출입 WHERE 차량구분 = :g', DBMS_SQL.NATIVE);
FOR i IN 1 .. v_gubun.COUNT LOOP
    DBMS_SQL.BIND_VARIABLE(c, ':g', v_gubun(i)); -- 값만 교체
    d := DBMS_SQL.EXECUTE(c); -- 파싱 없이 실행만 반복
    -- DBMS_SQL.COLUMN_VALUE(c, 1, v_cnt); ...
END LOOP;
DBMS_SQL.CLOSE_CURSOR(c);

-- 방식 (나) : EXECUTE IMMEDIATE - 차량구분 값을 리터럴로 결합
FOR i IN 1 .. v_gubun.COUNT LOOP
    v_sql := 'SELECT COUNT(*) FROM 차량출입 WHERE 차량구분 = ''' || v_gubun(i) || '''';
    EXECUTE IMMEDIATE v_sql INTO v_cnt; -- 값마다 텍스트가 달라짐
END LOOP;

-- 참고: 배열에 서로 다른 차량구분 코드가 N종 들어 있음
```

- ① 방식 (나)는 차량구분 리터럴만 다르고 SELECT 구조·공백·대소문자가 모두 같으므로, 기본 커서 공유 모드에서도 오라클이 리터럴을 바인드처럼 자동 치환해 하나의 부모 커서로 묶어 주어 반복 하드파싱이 일어나지 않는다.
- ② 방식 (가)는 DBMS_SQL.PARSE를 루프 밖에서 한 번만 호출하고 루프 안에서는 BIND_VARIABLE로 값만 바꿔 EXECUTE를 반복하므로, 파싱된 커서를 재사용해 반복 하드파싱을 피한다.
- ③ 방식 (나)는 값을 리터럴로 결합해 차량구분 코드 종류만큼 SQL 텍스트가 달라져 하드파싱이 되풀이되는데, USING 절로 바인딩했다면 텍스트가 :b 하나로 고정돼 커서를 공유했을 것이다.
- ④ DBMS_SQL이든 EXECUTE IMMEDIATE(NDS)든 바인드 변수를 쓰면 SQL 텍스트가 값과 무관하게 고정되어 라이브러리 캐시에서 커서를 공유하고, 사용자 입력을 구문으로 해석하지 않아 SQL Injection 위험도 줄어든다.

옵티마이저의 최적화 목표(전체범위 최적화 vs 부분범위 최적화)와, 통계정보가 비용 산정에 미치는 영향에 대한 설명으로 가장 적절한 것은?

- ① 옵티마이저의 최적화 목표(전체범위/부분범위)는 통계정보가 얼마나 정확한지에 따라 자동으로 정해진다. 행 수·NDV·히스토그램이 실제 분포와 맞으면 옵티마이저가 결과 집합의 크기를 미리 알아 스스로 목표를 고르므로, 통계를 정밀하게 수집해 두면 별도 설정 없이도 상황에 맞는 목표가 선택된다.
- ② 전체범위 최적화(ALL_ROWS)는 결과 집합 전체를 끝까지 반환하는 총비용을, 부분범위 최적화(FIRST_ROWS)는 앞쪽 일부 행을 빨리 내보내는 응답 시간을 목표로 삼는 서로 다른 최적화 목표이며, 어느 목표를 택하든 후보 계획의 비용은 통계정보를 바탕으로 산정된다.
- ③ 전체범위 최적화로 세운 계획은 결과 전체의 총비용을 기준으로 삼으므로 통계가 부정확해도 계획 품질이 유지되지만, 부분범위 최적화로 세운 계획은 앞쪽 몇 행만 보고 판단하는 만큼 카디널리티 오차에 그대로 흔들린다. 그래서 통계가 낡았다면 ALL_ROWS 쪽이 안전한 선택이 된다.
- ④ 통계가 최신일수록 옵티마이저는 부분범위 최적화를 선호해 첫 행을 더 빨리 내보내는 계획을 택한다. 실제 행 수를 알면 전체를 다 만들어 둘 필요가 없다고 판단하기 때문이므로, 응답 시간을 줄이려면 통계 최신화가 곧 부분범위 최적화를 켜는 수단이 된다.

판매전표 테이블에 하루치 마감분 5천만 건을 대량 INSERT하는 배치가 있다. 이 테이블에는 비고유 인덱스 4개, 기본키(PK) 1개, 외래키(FK) 1개, 그리고 행마다 이력을 남기는 AFTER INSERT 트리거가 걸려 있다. 적재가 갈수록 느려져 아래처럼 인덱스 유지 비용을 줄이는 전략을 검토한다. 대량 INSERT의 DML 부하와 그 완화 방안에 대한 설명으로 가장 적절하지 않은 것은?

아래

— 적재 전 : 비고유 인덱스를 UNUSABLE로 전환(엔트리 유지 중단)

```
ALTER INDEX ix_pt_고객 UNUSABLE;
ALTER INDEX ix_pt_상품 UNUSABLE;
ALTER INDEX ix_pt_점포 UNUSABLE;
ALTER INDEX ix_pt_일자 UNUSABLE;
```

— 대량 적재 (인덱스 유지 없이 테이블만 채움)

```
INSERT INTO 판매전표 SELECT * FROM 마감스테이징;
```

— 적재 후 : 인덱스 일괄 재생성

```
ALTER INDEX ix_pt_고객 REBUILD;
ALTER INDEX ix_pt_상품 REBUILD;
ALTER INDEX ix_pt_점포 REBUILD;
ALTER INDEX ix_pt_일자 REBUILD;
```

— 참고: PK/FK 제약, AFTER INSERT 트리거는 그대로 활성 상태

- ① 판매전표에 비고유 인덱스 4개·PK·FK·트리거가 걸려 있으면, 행마다 인덱스 엔트리 추가·제약 검증·트리거 실행이 INSERT에 얹혀 순수 데이터 적재보다 DML 부하가 커진다.
- ② 적재 전 비고유 인덱스를 UNUSABLE로 돌리고 적재 후 REBUILD하면, 적재 도중 인덱스 엔트리를 정렬·삽입하는 유지 비용과 인덱스 관련 redo가 줄어 전체 적재 시간을 단축할 수 있다.
- ③ 일반 INSERT는 테이블 블록뿐 아니라 딸린 인덱스 블록도 함께 변경하므로 그만큼 redo·undo가 늘며, 인덱스 개수가 많을수록 발생량이 증가한다.
- ④ Direct Path Insert(/*+ APPEND */)는 언두를 크게 남기고 인덱스도 행 단위로 실시간 갱신하므로, 인덱스가 많은 테이블일수록 일반 INSERT보다 DML 부하가 항상 더 크다.

교통카드 승하차이력 테이블은 이용일시(DATE)를 기준으로 월별 RANGE 파티셔닝되어 있고, 그 위에 두 개의 로컬 인덱스가 있다(아래 DDL). ix_loc_pfx는 선두 칼럼이 파티션 키(이용일시)인 로컬 프리픽스드(prefixed), ix_loc_npfx는 선두 칼럼이 카드번호여서 파티션 키를 앞세우지 않은 로컬 넌프리픽스드(nonprefixed)이다. 두 로컬 인덱스의 파티션 프루닝·스캔 범위에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
CREATE TABLE 승하차이력 (
  이용일시    DATE,
  카드번호    VARCHAR2(16),
  노선번호    VARCHAR2(6),
  정류장코드  VARCHAR2(8),
  요금        NUMBER
)
PARTITION BY RANGE (이용일시) (
  PARTITION p_2506 VALUES LESS THAN (DATE '2025-07-01'),
  PARTITION p_2507 VALUES LESS THAN (DATE '2025-08-01'),
  PARTITION p_2508 VALUES LESS THAN (DATE '2025-09-01'),
  PARTITION p_max  VALUES LESS THAN (MAXVALUE)
);

-- ix_loc_pfx : 로컬, 선두가 파티션 키(이용일시) → prefixed
CREATE INDEX ix_loc_pfx ON 승하차이력 (이용일시, 노선번호) LOCAL;

-- ix_loc_npfx : 로컬, 선두가 카드번호(파티션 키 아님) → nonprefixed
CREATE INDEX ix_loc_npfx ON 승하차이력 (카드번호, 이용일시) LOCAL;
```

- ① ix_loc_pfx는 선두가 파티션 키(이용일시)라, WHERE 이용일시 >= DATE '2025-08-01' 같은 날짜 조건이 오면 Partition Pruning으로 대응 인덱스 파티션만 스캔한다.
- ② ix_loc_npfx로 WHERE 카드번호 = :1처럼 카드번호만으로 조회하면, 그 카드의 이력이 어느 월 파티션에 흩어져 있는지 알 수 없어 모든 로컬 인덱스 파티션을 프로빙해야 한다.
- ③ ix_loc_npfx도 로컬 인덱스이므로 카드번호만으로 조회해도 프루닝이 걸려 단일 인덱스 파티션만 접근하며, 그 스캔 효율은 프리픽스드 로컬 인덱스와 동일하다.
- ④ ix_loc_npfx라도 WHERE 카드번호 = :1 AND 이용일시 >= DATE '2025-08-01'처럼 파티션 키 조건을 함께 주면, 그제야 프루닝이 적용돼 접근하는 인덱스 파티션 수가 줄어든다.

고속도로 통행료 정산 배치에서 통행내역 테이블(약 2억 4천만 건)과 요금소 테이블(약 1,500건)을 요금소코드로 병렬 해시 조인한 뒤 요금소코드별로 통행료를 집계한다. 두 테이블 모두 파티셔닝돼 있지 않다. 이 병렬 실행계획은 조인을 위해 두 테이블을 병렬 서버(Slave) 사이에 어떻게 분배하는가? 가장 정확히 지목한 것은?

아래

Id	Operation	Name	TQ	IN-OUT
0	SELECT STATEMENT			
1	PX COORDINATOR			
2	PX SEND QC (RANDOM)	:TQ10001	Q1,01	P->S
3	HASH GROUP BY		Q1,01	PCWP
4	PX RECEIVE		Q1,01	PCWP
5	PX SEND HASH	:TQ10000	Q1,00	P->P
6	HASH GROUP BY		Q1,00	PCWP
* 7	HASH JOIN		Q1,00	PCWP
8	TABLE ACCESS FULL	요금소	Q1,00	PCWP
9	PX BLOCK ITERATOR		Q1,00	PCWC
* 10	TABLE ACCESS FULL	통행내역	Q1,00	PCWP

- ① 통행내역(2억 4천만 건)이 각 병렬 서버에 통째로 REPLICATE(복제)되어 Id 10에서 서버마다 전량이 읽히고, 요금소(1,500건)는 Id 9 PX BLOCK ITERATOR로 블록 범위를 나눠 병렬 스캔된다. 조인(Id 7)은 이렇게 갖춰진 두 입력으로 슬레이브 안에서 수행되고, Id 5 PX SEND HASH는 그 조인 결과를 상위로 넘기는 경로다.
- ② 한 병렬 서버가 요금소 1,500건을 읽어 Id 8 위의 PX SEND BROADCAST로 전 서버에 뿌리고, 통행내역은 Id 9 PX BLOCK ITERATOR로 재분배 없이 읽는 broadcast 방식이다. Id 5 PX SEND HASH의 :TQ10000이 그 브로드캐스트 경로이며, 큰 통행내역은 네트워크로 흐르지 않는다.
- ③ 요금소와 통행내역을 양쪽 다 요금소코드 해시로 재분배하는 hash-hash 방식이며, Id 5 PX SEND HASH가 :TQ10000으로 그 재분배를 수행하고 Id 4 PX RECEIVE가 Q1,01에서 받아 짝을 맞추므로, 두 테이블의 같은 요금소코드가 같은 슬레이브로 모인 뒤 Id 7에서 조인된다.
- ④ 요금소는 재분배도 브로드캐스트도 없이 각 병렬 서버가 자체적으로 전체를 Full Scan(Id 8)하는 REPLICATE 방식이고, 통행내역은 각 서버가 Id 9 PX BLOCK ITERATOR로 자기 블록 범위만 병렬 스캔(Id 10)한다. 조인은 재분배 없이 슬레이브 안에서 수행되며, Id 5 PX SEND HASH는 상위 HASH GROUP BY 재분배용이다.

분석함수(윈도우 함수)의 프레임과 정렬, 그리고 집합 연산자의 정렬·중복 제거에 대한 설명으로 가장 적절한 것은?

- ① 한 SELECT에 분석함수가 여러 개 있으면 정렬 기준과 무관하게 하나의 Window Sort로 묶여 처리되므로, 분석함수마다 ORDER BY 기준이 서로 달라도 정렬은 한 번이면 충분하다.
- ② UNION ALL은 중복 제거 정렬(Sort Unique) 없이 두 집합을 이어 붙이는 연산이므로, 그 위에 ORDER BY를 덧붙여 전체 결과의 정렬을 요구해도 최종 실행계획에는 정렬 오퍼레이션이 추가되지 않는다.
- ③ 윈도우 프레임을 ROWS로 지정하면 물리적 행 개수 단위로, RANGE로 지정하면 ORDER BY 값이 같은 행을 하나의 논리적 범위로 묶어 집계하므로, 정렬 기준 값에 동순위(ties)가 있으면 같은 입력이라도 두 프레임의 누적 결과가 달라질 수 있다.
- ④ RANK·ROW_NUMBER 같은 순위 함수는 ORDER BY로 정렬이 필요하지만, SUM·COUNT 같은 집계형 분석함수는 순위를 매기지 않으므로 ORDER BY를 함께 써도 정렬 없이 누적을 계산한다.

19

SQL Server에서 공연예매 현황 테이블의 공연회차 'S7' 행(초기 잔여좌석 = 20)을 두 세션이 아래 순서로 다룬다. TX1은 t1에서 잔여좌석을 6 줄이고(아직 커밋하지 않음), TX2는 t2에서 같은 조건의 SELECT로 (가)를 읽는다. 이후 t3에서 TX1이 ROLLBACK한다. TX2의 격리수준은 READ UNCOMMITTED이다. TX2가 t2에서 읽은 값 (가)와 그때 발생하는 현상에 대한 진단으로 가장 적절한 것은? (단, 발문에 없는 힌트·격리수준 조정은 하지 않았고, TX1의 감소분은 t3에서 그대로 롤백된다.)

아래

시각	TX1 (예매 반영, 미완결)	TX2 (현황 조회, READ UNCOMMITTED)
t1	UPDATE 공연예매 SET 잔여좌석 = 잔여좌석 - 6 WHERE 공연회차='S7'; (→ 14, COMMIT 안 함)	
t2		SELECT 잔여좌석 FROM 공연예매 WHERE 공연회차='S7'; → (가) 읽음
t3	ROLLBACK; (→ 20 복원)	

초기 잔여좌석(S7) = 20

- TX2가 READ UNCOMMITTED라 t2에서 아직 커밋되지 않은 14를 읽어 (가)=14이며, TX1이 t3에서 롤백하므로 실제로 확정된 적 없는 값을 읽은 Dirty Read다. 격리수준을 READ COMMITTED 이상으로 올리면 t2는 커밋된 20을 보게 되어 이 현상이 사라진다.
- (가)=20이다. READ UNCOMMITTED는 공유 락을 걸지 않을 뿐 SELECT는 커밋된 값만 읽으므로, t2 시점에 TX1의 미커밋 14는 보이지 않고 t3에서 복원되는 값과 같은 20을 반환한다. 읽은 값이 확정값과 일치하므로 이상현상은 발생하지 않는다.
- (가)=14인 것은 맞지만 이는 Non-Repeatable Read다. 초기 20이 t1의 UPDATE로 14가 되었고 t2가 그 달라진 값을 읽었으니 같은 행에서 20과 14라는 두 값이 관측된 셈이기 때문이며, 격리수준을 REPEATABLE READ로 올려야 사라진다.
- TX2가 t2에서 락을 기다리지 않고 곧바로 S7 행의 14를 읽었다는 사실이 이 현상이 Dirty Read가 아님을 보여 준다. READ COMMITTED에서도 SELECT는 대기 없이 읽으므로, 대기 없이 읽었다는 것만으로 t1의 미커밋 UPDATE와 무관한 정상 읽기로 보아야 한다.

20

배차 관제 화면에서 세션 27이 특정 배차요청 한 건을 SELECT ... FROM 배차요청 WHERE 배차번호 = 7180 FOR UPDATE로 잠근 뒤 아직 커밋하지 않았다. 그 사이 세션 38이 같은 배차번호(7180) 행을 다시 다루려 한다. 이때 v\$sqllock을 조회한 결과가 아래와 같다. SELECT ... FOR UPDATE가 건 행 잠금(TX 락)의 획득·대기·오픈 동작에 대한 해석으로 가장 적절하지 않은 것은?

아래

[v\$sqllock - 배차번호 7180 행에 걸린 TX(트랜잭션) 락]

SID	TYPE	ID1	LMODE	REQUEST	BLOCK
27	TX	393241	6	0	1
38	TX	393241	0	6	0

- * TX 모드 코드: 6 = X(Exclusive, 행 배타)
- * SID 27 : SELECT ... FOR UPDATE 로 해당 행에 X(6) 보유 (BLOCK=1)
- * SID 38 : 같은 행을 잠그려 X(6)를 REQUEST 하며 대기

- 세션 27은 SELECT ... FOR UPDATE로 배차번호 7180 행에 TX 락을 X(6) 모드로 획득했고, BLOCK=1로 같은 행을 원하는 다른 세션을 막고 있다. 이는 테이블 전체가 아니라 그 행에 대한 잠금이다.
- 세션 38이 같은 행을 기본 SELECT ... FOR UPDATE(WAIT, 무한 대기)로 잠그려 하면 X(6)를 REQUEST한 채, 세션 27이 커밋하거나 롤백해 락을 놓을 때까지 대기한다.
- 세션 38이 SELECT ... FOR UPDATE NOWAIT를 쓰면 세션 27이 이미 잠근 행이라도 대기 없이 즉시 그 행의 락을 획득해 곧바로 갱신을 이어간다.
- 세션 38이 SELECT ... FOR UPDATE SKIP LOCKED를 쓰면 세션 27이 잠근 7180 행은 건너뛰고, 잠기지 않은 다른 배차요청 행만 잠가 대기 없이 처리 대상을 확보한다.

정답 및 해설

■ 정답 모아보기

1. ②	2. ②	3. ④	4. ②	5. ③
6. ①	7. ①	8. ④	9. ①	10. ④
11. ②	12. ③	13. ①	14. ②	15. ④
16. ③	17. ④	18. ③	19. ①	20. ③

문제 1. 정답 ②

I/O 효율화가 겨누는 것은 느린 물리적 I/O(디스크 읽기)를 줄이는 것입니다. 방향을 정리하면 이렇습니다.

논리적 I/O : 블록을 읽어 달라고 요청한 총량 (버퍼 캐시 조회 포함)

물리적 I/O : 그 요청 중 캐시에 없어 디스크에서 실제로 읽어 온 양 (가장 비쌌)

효율화 방향 : 논리적 I/O 자체를 줄이고(불필요한 블록 접근 제거),

그중에서도 디스크를 때리는 물리적 I/O를 최대한 줄인다.

버퍼 캐시 : 적중률이 높을수록 → 캐시에서 해결 → 물리적 I/O는 '줄어든다'

②는 이 방향을 정반대로 뒤집었습니다. "물리적 I/O를 늘려서라도 논리적 I/O를 줄인다"는 우선순위를 거꾸로 세운 서술이고(정작 가장 비싼 것은 물리적 I/O입니다), "적중률이 높을수록 물리적 I/O가 증가한다"는 것도 사실과 반대입니다. 캐시 적중률이 높다는 것은 디스크까지 내려가지 않고 캐시에서 해결되는 비율이 높다는 뜻이므로, 물리적 I/O는 줄어듭니다.

①·③·④는 옳습니다. Full Scan의 Sequential·Multiblock I/O로 인한 Call 절감(①), Prefetch가 대기를 겹쳐 지연을 감추는 효과(③), 인덱스 경유 Random 액세스의 건별 Single Block I/O 경향(④)이 모두 블록 I/O 효율화의 원리에 부합합니다.

선택지	판정	이유
①	○	Full Table Scan은 인접 블록을 저장 순서대로 훑는 Sequential 액세스라 Multiblock I/O와 맞물려, 같은 블록 수를 읽어도 I/O Call 횟수를 줄입니다
②	×	효율화의 초점은 가장 비싼 물리적 I/O를 줄이는 것이며, 캐시 적중률이 높을수록 물리적 I/O는 줄어듭니다. 우선순위와 적중률-물리 I/O 관계를 함께 뒤집은 서술입니다
③	○	Prefetch는 필요할 블록을 미리 요청해 개별 대기를 겹치게 함으로써 I/O 지연을 감추는 기법입니다
④	○	인덱스 ROWID로 흩어진 테이블 블록을 건별로 집어 오는 Random 액세스는 Single Block I/O 경향을 띠며, 연속 블록을 묶는 Sequential 액세스와 대비됩니다

■ 이 문제의 핵심

1. I/O 효율화의 최우선 표적은 물리적 I/O(디스크 읽기)입니다. 논리적 I/O를 줄이되, 그중에서도 물리적 I/O를 최대한 줄이는 것이 방향입니다.
2. 버퍼 캐시 적중률이 높을수록 물리적 I/O는 줄어듭니다. 캐시에서 해결되면 디스크까지 내려가지 않습니다.
3. Sequential 액세스 ↔ Multiblock I/O: 연속 블록을 묶어 읽어 대량 스캔의 I/O Call을 아깁니다.
4. Prefetch는 대기를 겹쳐 지연을 감추고, 인덱스 경유 Random 액세스는 Single Block I/O 경향을 띕니다.

■ 한 줄 정리 I/O 효율화는 가장 비싼 물리적 I/O를 줄이는 방향이고 캐시 적중률이 높을수록 물리적 I/O가 줄어드는데, "물리적 I/O를 늘려서라도 논리적 I/O를 줄이며 적중률이 높으면 물리 I/O가 는다"는 서술은 방향을 뒤집은 것입니다.

문제 2. 정답 ②

세 요소의 역할과 커밋 시점의 순서를 정리하면 이렇습니다.

Redo : 변경을 '재현(roll forward)'하는 정보 → 장애 후 커밋된 변경을 다시 적용

Undo : 변경을 '되돌리는(roll back)' 정보 → 롤백 + 다른 세션의 읽기 일관성(과거 이미지 재구성)

Write-Ahead Logging(로그 선기록)

규칙 : 데이터 블록을 데이터파일에 반영하기 '전에' 그 변경의 Redo를 로그에 먼저 기록

커밋 : LGWR가 해당 Redo를 로그 파일에 flush → 이 순간 영속성 확보

(더티 데이터 블록은 나중에 DBWR가 별도로 기록 - 커밋과 무관)

②는 이 구조를 정확히 짚었습니다. WAL은 로그를 데이터보다 먼저 남기는 규칙이고, 커밋의 완료는 데이터 블록 기록이 아니라 LGWR의 Redo flush로 보장됩니다. 그래서 커밋은 빠르고(로그만 내려쓰면 됨), 데이터 블록은 나중에 느긋하게 기록돼도 복구가 가능합니다.

①·③·④는 각 용어가 불러일으키는 반사적 연상을 사실로 옮긴 함정입니다.

선택지	판정	이유
①	×	"커밋=데이터가 디스크에 써진 상태"라는 연상이지만, WAL 덕분에 커밋 시점에 내려쓰지는 것은 Redo 로그(LGWR)이고 더티 데이터 블록은 나중에 DBWR가 기록합니다. DBWR→LGWR로 순서를 뒤집었고, 그래서 "커밋 시간이 더티 블록 수에 비례한다"는 귀결도 틀립니다. 커밋이 빠른 이유가 바로 로그만 내려쓰면 되기 때문입니다
②	○	데이터 블록 반영 전에 Redo를 로그에 먼저 남기는 것이 WAL이며, 커밋 시 LGWR가 Redo를 flush해 연속성을 확보합니다. 규칙과 커밋 동작을 정확히 서술했습니다
③	×	"Undo=롤백 전용"이라는 반사적 연상입니다. 과거 이미지를 재구성하는 재료는 Redo가 아니라 Undo이고, Redo는 거꾸로 훑는 정보가 아니라 앞으로 재적용하는 정보입니다. Undo 세그먼트도 커밋 즉시 해제되지 않고 진행 중인 일관성 읽기를 위해 일정 기간 보존됩니다
④	×	Redo와 Undo의 역할을 서로 맞바꿨습니다. Redo가 재적용(roll forward), Undo가 취소(rollback)입니다. 역할을 뒤집은 탓에 WAL까지 "Undo 선기록 규칙"으로 옮겨 놓았는데, 데이터 블록 반영 전에 먼저 남겨야 하는 것은 Undo가 아니라 Redo입니다

■ 이 문제의 핵심

1. Redo = 재적용(roll forward), Undo = 되돌리기(rollback). Undo는 롤백뿐 아니라 읽기 일관성(과거 이미지 재구성)에도 쓰입니다.
2. Write-Ahead Logging: 데이터 블록을 데이터파일에 반영하기 전에 그 변경의 Redo를 로그에 먼저 기록합니다.
3. 커밋의 연속성은 LGWR의 Redo flush로 보장됩니다. 더티 데이터 블록의 기록(DBWR)은 커밋과 분리되어 나중에 일어납니다.
4. Undo는 커밋과 동시에 버려지지 않고, 진행 중인 일관성 읽기를 위해 일정 기간 보존됩니다.

■ 한 줄 정리 WAL은 데이터보다 Redo를 먼저 로그에 남기는 규칙이고 커밋은 LGWR의 Redo flush로 완결되며, "커밋=데이터 선기록·Undo=롤백 전용·Redo와 Undo 역할 맞바꿈"은 이름에서 온 반사적 오해입니다.

문제 3. 정답 ④

TKPROF는 세션의 SQL을 비재귀(애플리케이션이 직접 던진 것)와 재귀(서버가 내부적으로 수행한 것)로 갈라 합산합니다. 함수 안의 SELECT는 PL/SQL에서 수행되므로 재귀 statement로 잡힙니다. 먼저 User Call과 Recursive Call을 쉼니다.

User Call(클라이언트 왕복) = 비재귀 블록의 DB Call
 = Parse 1 + Execute 1 + Fetch 301 = 303회
 Recursive Call(서버 내부) = 재귀 블록의 DB Call
 = Parse 1 + Execute 60,000 + Fetch 60,000 = 120,001회
 비율 = 120,001 / 303 ≈ 396배

DB Call은 120,304회지만, 그중 클라이언트를 실제로 오간 User Call은 303회뿐입니다. 재귀 120,001회는 함수가 행마다 서버 안에서 스스로 부른 것이라 네트워크 왕복을 만들지 않습니다. 비재귀 Fetch 301회는 6만 행을 배열로 실어 나른 왕복입니다.

비재귀 배열 크기 = 60,000 ÷ (301 - 1) = 200 행/Fetch ← User Call 측(왕복)
 재귀 수행 횟수 = 60,000 (함수가 행마다 1회) ← Recursive Call 측(내부 수행)
 재귀 1회당 Query = 480,000 ÷ 60,000 = 8 블록/수행

이제 시간의 정체를 봅니다.

비재귀 Elapsed = 3.512초 ← 애플리케이션 왕복·본 쿼리
 재귀 Elapsed = 46.501초 ← 응답시간의 대부분

병목 46.5초는 재귀 수행 측(Recursive Call)에 있고, User Call 왕복 측(303회, 3.5초)과는 독립입니다.

측 A: User Call(왕복 303회) — 배열 인출 크기로 조절, 이미 3.5초로 작음
 측 B: Recursive Call(60,000회) — 행별 함수 호출로 발생, 46.5초 정체의 정체

두 측은 독립: 배열 크기를 키워도 측 A의 301회만 줄 뿐 측 B의 60,000회 재귀는 그대로.

따라서 처방은 배열 크기(측 A)나 버퍼 캐시가 아니라, 행마다 도는 재귀 SQL 자체를 없애는 것(조인 전환·스칼라 서브쿼리 캐싱)입니다. ④가 두 측을 옳게 분리했습니다.

선택지	판정	이유
①	×	재귀 Execute·Fetch 각 60,000회는 서버 내부 Recursive Call이라 클라이언트 왕복이 아닙니다. 배열 크기 200행은 비재귀 Fetch 301회(User Call) 축에만 걸리므로, 2,000행으로 키워도 줄어드는 것은 301회뿐이고 재귀 46.501초는 그대로 남습니다
②	×	DB Call 120,304회라는 합산은 맞지만 User Call은 비재귀 303회뿐이고 나머지 120,001회는 왕복 없는 Recursive Call입니다. 전부 User Call로 본 것은 데이터베이스 Call과 User Call을 같은 축으로 뭉겐 오류라, 커넥션 풀링·세션 재사용은 46.501초를 겨냥하지 못합니다
③	×	재귀 Query 480,000이 비재귀 5,000의 96배인 것은 맞지만, 그것은 함수가 60,000회 수행되며 회당 8블록씩 쌓인 결과입니다. 버퍼 캐시는 물리 읽기 Disk 4,000을 줄일 뿐 논리 읽기와 60,000회 수행을 없애지 못하므로, 정제 원인을 논리 읽기로 오귀속한 것입니다
④	○	User Call은 비재귀 303회, Recursive Call은 재귀 120,001회로 두 축이 분리되고, 46.5초 병목은 60,000회 재귀 수행 축에 있어 배열 크기가 아니라 행별 재귀 SQL 제거로만 줄어든다는 진단이 정확합니다

■ 이 문제의 핵심

1. User Call(왕복)과 DB Call·Recursive Call은 다른 축입니다. User Call은 비재귀 블록의 303회(Parse 1+Execute 1+Fetch 301)뿐이고, 재귀 120,001회는 서버 내부 수행이라 네트워크 왕복을 만들지 않습니다.

2. 함수/PL/SQL 안의 SELECT는 재귀 statement로 집계됩니다. 행마다 호출되면 재귀 Execute·Fetch가 행 수(60,000)만큼 발생합니다.

3. 병목 46.5초는 재귀 수행 축에 있고, 배열 인출 크기(User Call 축, 303회·3.5초)와 독립입니다. 축을 인과로 묶으면 배열 크기·커넥션 풀링·버퍼 캐시로 헛짚습니다.

4. 처방은 행별 재귀 SQL을 없애는 것(조인 전환·스칼라 서브쿼리 캐싱)이지, 왕복·물리 읽기 조절이 아닙니다.

■ 한 줄 정리 DB Call 120,304회 중 User Call은 비재귀 303회뿐이고 재귀 120,001회는 왕복 없는 서버 내부 수행이므로, 46.5초 병목은 배열 크기가 아니라 행별 재귀 SQL 제거로만 줄어듭니다.

문제 4. 정답 ②

응답 시간 분석의 뼈대는 시간을 두 조각으로 가르는 것입니다.

응답 시간(Response Time) = Service Time + Wait Time

Service Time : CPU를 실제로 쓴 시간 ← 'CPU Time'이 곧 Service Time (같은 것)

Wait Time : 자원(블록·락·로그 등)을 기다린 시간

DB Time = 모든 세션의 (CPU 소비 + 대기)를 합산한 값

→ 세션이 여러 개면 wall-clock 경과 시간을 훨씬 넘어설 수 있다 (같지 않음)

Top Timed Events : 상위 이벤트가 DB Time에서 차지하는 '비중'으로 병목의 성격을 판별

②는 이 구조를 정확히 짚었습니다. 응답 시간을 Service(=CPU) Time과 Wait Time으로 나누고, Top Timed Events의 비중으로 병목이 CPU 쪽인지 특정 대기 쪽인지 읽는 것이 AWR/ASH 해석의 정석입니다.

①·③·④는 각각 개념의 경계를 흐린 함정입니다.

①은 Service Time과 CPU Time을 서로 다른 항인 양 따로 더하게 했습니다. 둘은 같은 것(서비스 시간 = CPU 소비 시간)이므로 이렇게 더하면 CPU 시간을 이중 계산합니다.

③은 DB Time을 wall-clock 경과 시간과 같다고 했지만, DB Time은 세션들의 시간을 합산한 값이라 경과 시간을 넘어설 수 있습니다. 몇 배로 찍히는 것은 스냅샷 구간 오류가 아니라 동시 세션이 많았다는 정상 신호이고, 비중도 DB Time으로 나눠 읽습니다.

④는 CPU를 쓴 시간(Service Time)을 응답 시간에서 빼 버렸습니다. CPU 시간도 응답 시간의 한 축이라 **DB CPU** 행이야말로 CPU 바운드인지 가르는 근거입니다.

선택지	판정	이유
①	×	Service Time과 CPU Time은 같은 것(서비스 시간 = CPU 소비 시간)입니다. Top Timed Events의 DB CPU 가 곧 그 서비스 시간이므로, 다른 이름을 별개 항목으로 놓고 세 항목을 더하면 CPU 시간을 이중 계산하게 됩니다
②	○	응답 시간을 Service(=CPU) Time과 Wait Time으로 가르고, Top Timed Events의 DB Time 비중으로 병목의 성격을 판별하는 것이 AWR 해석의 정석입니다
③	×	DB Time은 모든 세션의 CPU+대기를 합산한 값이라 두 스냅샷 사이 경과 시간을 넘어설 수 있습니다. 경과 시간의 몇 배로 찍히는 것은 스냅샷 오류가 아니라 동시 세션이 많았다는 신호이고, 이벤트 비중도 경과 시간이 아니라 DB Time으로 나눠 읽습니다
④	×	CPU를 쓴 Service Time도 응답 시간의 한 축입니다. 이름에 대기가 든 이벤트만 응답 시간에 넣으면 CPU 시간이 통째로 빠지고, DB CPU 행을 병목 판단에서 빼면 CPU 바운드 상황을 아예 못 잡습니다

■ 이 문제의 핵심

1. 응답 시간 = Service Time + Wait Time. Service Time은 곧 CPU Time이므로 둘을 별개 항목으로 나눠 더하면 이중 계산입니다.
2. DB Time은 세션별 CPU+대기의 합산값이라, 세션이 여러 개면 wall-clock 경과 시간을 넘어설 수 있습니다.
3. Top Timed Events의 DB Time 비중으로 병목이 CPU 쪽인지 특정 대기 쪽인지 가늠합니다.
4. CPU를 쓴 시간도 응답 시간의 한 축이므로, 대기 이벤트만 응답 시간으로 보면 안 됩니다.

■ 한 줄 정리 응답 시간은 Service(=CPU) Time과 Wait Time의 합이고 Top Timed Events의 DB Time 비중으로 병목을 읽으며, Service Time을 CPU Time과 별개 항목으로 더하는 것은 같은 것을 다른 이름으로 이중 계산하는 오류입니다.

문제 5. 정답 ③

먼저 BCHR로 논리 대비 물리 읽기 비율을 확인합니다.

$$\begin{aligned} \text{BCHR} &= ((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100 \\ &= (2,000,000 + 0 - 300,000) / 2,000,000 \times 100 \\ &= 1,700,000 / 2,000,000 \times 100 = 85.0\% \end{aligned}$$

$$\text{디스크 물리 읽기 비중} = 300,000 / 2,000,000 \times 100 = 15.0\%$$

논리 200만 블록 중 15%를 디스크에서 물리적으로 읽었습니다. 다음은 시간의 정체입니다.

$$\begin{aligned} \text{CPU 비중} &= 7.000 / 55.000 \times 100 \approx 12.7\% \quad \rightarrow \text{나머지 약 87.3\%가 대기} \\ \text{Elapsed} - \text{CPU} &= 55.000 - 7.000 = 48.0\text{초} \quad \leftarrow \text{일한 시간이 아니라 기다린 시간} \end{aligned}$$

응답시간의 대부분이 대기이고, 그 대기가 어디서 났는지는 Row Source가 지목합니다.

$$\begin{aligned} \text{Full Scan cr 비중} &= 2,000,000 / 2,000,000 = 100\% \quad \leftarrow \text{논리 읽기 전량} \\ \text{Full Scan pr 비중} &= 300,000 / 300,000 = 100\% \quad \leftarrow \text{물리 읽기 전량} \\ \text{Full Scan time} &= 53.0\text{초} \quad (\text{전체 Elapsed 55초의 대부분}) \end{aligned}$$

$$\begin{aligned} \text{SORT} \cdot \text{GROUP BY 자체 증분 시간} &= 54.9 - 53.0 \approx 1.9\text{초} \quad \leftarrow \text{정렬} \cdot \text{집계는 작음} \\ \text{SORT} \cdot \text{GROUP BY 자체 증분 cr} &= 2,000,000 - 2,000,000 = 0 \end{aligned}$$

블록 소비(cr)·물리 읽기(pr)·소요 시간(time)이 세 지표 모두 게임접속로그 Full Scan 한 노드에 몰려 있습니다. SORT의 **pw=0**이라 정렬 디스크 스필도 없습니다. 따라서 병목은 1억 2천만 건 Full Scan I/O이며, ③이 BCHR 해석과 병목 지목을 모두 옳게 했습니다.

선택지	판정	이유
①	×	CPU 7초는 Elapsed 55초의 12.7%뿐이라 CPU 바운드가 아닙니다. pw=0 이 참인 것과 CPU 바운드는 별개이고, 응답시간의 약 87.3%(약 48초)는 Full Scan I/O 대기입니다. DOP를 4로 올려도 줄어드는 것은 7초 쪽이라 원인 오귀속입니다
②	×	Row Source의 time은 누적이라 SORT ORDER BY의 54.9초에는 그 아래 Full Scan 53초가 이미 들어 있습니다. SORT·GROUP BY의 증분 시간은 약 1.9초(54.9-53.0), 증분 cr은 0이므로, 정렬 인덱스는 48초 대기를 겨냥하지 못합니다
③	○	(2,000,000-300,000)/2,000,000 = 85.0%로 BCHR이 정확하고, cr(100%)·pr(100%)·time(53초)이 모두 게임접속로그 Full Scan에 집중되어 병목 지목이 맞습니다
④	×	SORT ORDER BY의 pw=0 이라 정렬 디스크 스필은 없습니다. pr 300,000은 스필이 아니라 Full Scan의 물리 읽기이고 SORT의 54.9초도 그 Full Scan 53초를 물려받은 누적값이므로, PGA 확대는 48초 대기를 겨냥하지 못한 헛짓은 처방입니다

■ 이 문제의 핵심

1. BCHR = ((Query+Current)-Disk)/(Query+Current) × 100. 여기서는 (2,000,000-300,000)/2,000,000 = 85.0%입니다.
2. CPU 비중(12.7%)이 낮으면 CPU 바운드가 아닙니다. Elapsed-CPU(48초)가 대기이고, 그 정체는 Row Source의 **pr·time**으로 특정 노드를 지목해야 드러납니다.
3. 병목 노드는 cr·pr·time이 가장 큰 곳 — 여기서는 게임접속로그 Full Scan(cr 100%, pr 100%, time 53초)입니다. 상위 SORT·GROUP BY의 증분 cr은 0, 증분 time은 약 1.9초뿐입니다.
4. **pw=0**은 정렬 디스크 스푼이 없다는 신호 — PGA 부족을 원인으로 오귀속하지 않도록 반드시 확인합니다.
5. 결과 행수(3,000)가 적어도 스캔량은 클 수 있습니다. 출력이 작다고 Full Scan 1억 2천만 건의 I/O가 작아지지 않습니다.
 - 한 줄 정리 BCHR 85%와 CPU 12.7%까지 옮겨 읽어도, cr·pr·time이 모두 게임접속로그 Full Scan에 몰리고 pw=0인 사실을 놓치면 병목을 CPU·정렬 인덱스·PGA 스푼으로 오귀속하게 됩니다.

문제 6. 정답 ①

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 총전_IX Card 9,000 ← 인덱스에서 훑고 남은 건수
TABLE ACCESS BY INDEX ROWID 총전세션 Card 2,700 ← 테이블까지 거친 뒤 남은 건수

총전_IX는 (총전소코드, 총전시작일시) 순서입니다. WHERE에는 총전소코드 = 'C15'(등치)와 총전시작일시 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 연속해서 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 9,000건입니다.

결제방식은 **총전_IX**의 구성 칼럼이 아닙니다. 인덱스에 값이 없으니 인덱스 단계에서는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 9,000건이 이 단계를 거치며 2,700건으로 줄어듭니다.

9,000 (인덱스 액세스: 총전소코드=, 총전시작일시 범위)
└─ 테이블 필터(결제방식='앱') 적용 → 2,700 (최종)

즉 ①이 이 구조를 정확히 서술합니다. 결제방식을 인덱스 필터로 오해한 ②, 선두 칼럼 등치까지 필터로 밀어 넣은 ③, 인덱스 반환 건수를 최종 건수로 본 ④는 모두 이 Card 흐름과 어긋납니다.

선택지	판정	이유
①	○	총전소코드(등치)·총전시작일시(범위)는 인덱스 액세스 조건, 결제방식은 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 9,000→2,700으로 줄어드는 흐름과 일치합니다
②	×	결제방식은 총전_IX 구성 칼럼이 아니므로 인덱스 필터조차 될 수 없습니다. 그것이 스캔 범위를 좁혔다면 Id 3의 Card가 이미 2,700이었을 텐데 실제로는 9,000이고, Cost 22는 두 칼럼만으로 훑은 좁은 구간의 비용입니다
③	×	두 번째 칼럼이 범위라는 성질을 선두 칼럼에까지 전제로 확대한 진술입니다. 선두 칼럼 등치는 그 자체로 액세스 조건이고, 범위는 그 다음 칼럼부터 적용됩니다. 870만 건을 통째로 훑었다면 Id 3의 Cost가 22에 그칠 수 없고 오퍼레이션도 FULL SCAN으로 잡혔을 것입니다
④	×	9,000은 인덱스 단계 Card일 뿐이고, 결제방식 필터를 거친 Id 2의 Card가 이미 2,700입니다. 감소가 Id 1 조인에서 일어났다는 것도 어긋납니다 — 총전소_PK는 유니크 조인이라 Id 1의 Card는 Id 2의 2,700을 그대로 유지합니다

▪ 이 문제의 핵심

1. 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 총전소코드(=)·총전시작일시(범위)가 INDEX RANGE SCAN 구간을 만듭니다(Card 9,000).
2. 인덱스에 없는 칼럼은 테이블 필터. 결제방식은 ROWID로 테이블을 읽은 뒤 TABLE ACCESS BY INDEX ROWID 단계에서 걸러집니다.
3. 두 노드의 Card 차이(9,000 vs 2,700)가 곧 테이블 필터의 효과입니다. 인덱스 반환 건수 ≠ 최종 건수.
4. 범위 칼럼의 성질을 선두 등치 칼럼에까지 밀지 않습니다. 선두 등치는 액세스 조건이고, "범위 뒤는 필터"는 그 다음 칼럼에만 걸립니다.
 - 한 줄 정리 선두 칼럼 등치(총전소코드)와 다음 칼럼 범위(총전시작일시)는 인덱스 액세스 조건이 되고, 인덱스에 없는 결제방식은 테이블 액세스 단계 필터가 되어 Card가 9,000에서 2,700으로 줄어듭니다.

문제 7. 정답 ①

Row Source의 cr는 누적이므로, 테이블 액세스 노드의 191,400에서 자식 인덱스 1,400을 빼면 테이블 자체 블록 I/O가 나옵니다. 이를 ROWID 수로 나눠 클러스터링 팩터를 가늠합니다.

테이블 자체 블록 I/O = 191,400 - 1,400 = 190,000

액세스 ROWID(행) 수 = 200,000

CF 추정 = 190,000 / 200,000 ≈ 0.95 블록/행

→ 행마다 거의 다른 블록 → 클러스터링 팩터 최악(≈ 행 수에 근접)

CF가 나쁘면 인덱스로 뽑은 각 행이 서로 다른 테이블 블록에 흩어져 있어, 테이블 Random 액세스가 한 행당 블록 하나씩을 단일 블록으로 읽습니다. 어디에 시간이 쓸렸는지 봅시다.

테이블 액세스 cr 비중 = 190,000 / 191,400 × 100 ≈ 99.3% ← 논리 읽기의 대부분

인덱스 스캔 cr 비중 = 1,400 / 191,400 × 100 ≈ 0.7% ← 극소

테이블 액세스 time = 24.5초 (인덱스 스캔 time은 0.35초)

이제 손익분기점입니다. 테이블은 약 40만 블록인데, 인덱스 경유로 읽는 테이블 블록이 190,000입니다.

인덱스 경유 테이블 블록 = 190,000

테이블 총 블록 = 40,000,000 / 100 = 400,000

비율 = 190,000 / 400,000 = 47.5%

→ 테이블의 절반 가까이를 '단일 블록 랜덤 읽기'라는 느린 경로로 훑음

→ 다중 블록 Full Scan 손익분기점을 이미 넘음

인덱스 스캔(cr 1,400·0.35초)은 문제가 아닙니다. 정체는 CF 최악으로 부풀어 오른 테이블 Random 액세스 190,000블록이며, 이는 필요한 컬럼을 인덱스에 더한 커버링 인덱스로 테이블 액세스 자체를 없애거나 Full Scan으로 전환할 때 사라집니다. 따라서 ①이 원인을 정확히 지목했습니다.

선택지	판정	이유
①	○	테이블 자체 cr 190,000(=191,400-1,400)이 ROWID 200,000과 거의 같아 CF≈0.95로 최악이고, 이 190,000이 cr의 99.3%·24.5초를 차지하며 40만 블록의 47.5%를 랜덤 읽기로 훑어 손익분기점을 넘었다는 지목과 커버링 인덱스 처방이 정확합니다
②	×	cr 1,400·pr 200·time 0.35초는 맞지만, 그 값이 곧 INDEX RANGE SCAN이 병목이 아니라는 증거입니다(cr의 0.7%). 리빌드는 리프를 조밀하게 만들 뿐 테이블 블록의 물리적 배치(CF)를 바꾸지 못하므로, 190,000블록의 테이블 Random 액세스는 그대로 남습니다
③	×	Fetch 401회는 배열 크기 500(=200,000/400)의 결과일 뿐입니다. 24.5초를 401로 나눠 61밀리초를 왕복 지연이라 부른 것은 순환 논증이고, 그 24.5초는 Row Source가 테이블 액세스 노드에 붙여 놓은 시간입니다. Fetch를 41회로 줄여도 읽어야 할 190,000블록은 그대로입니다
④	×	Disk 40,000이 24초를 만들었다면 0.6밀리초라는 계산은 성립하지만, 40,000을 전부 캐시 적중으로 돌려도 테이블 자체 논리 읽기 190,000블록(단일 블록 랜덤)은 남습니다. 병목은 물리 읽기가 아니라 CF가 만든 논리 블록 액세스 횟수이므로 버퍼 캐시로 오귀속한 것입니다

■ 이 문제의 핵심

1. Row Source cr는 누적입니다. 테이블 자체 블록 I/O = 상위(191,400) - 자식 인덱스(1,400) = 190,000. 이걸 ROWID 수로 나눠 CF를 가늠합니다(190,000/200,000 ≈ 0.95 블록/행).
2. CF가 최악이면 테이블 Random 액세스가 병목입니다. 190,000이 cr의 99.3%·time 24.5초를 차지하고, 인덱스 스캔(1,400·0.35초)은 무죄입니다.
3. 손익분기점: 인덱스 경유 테이블 블록 190,000이 총 40만 블록의 47.5%를 단일 블록 랜덤으로 훑으면 다중 블록 Full Scan보다 불리합니다.
4. 처방은 인덱스 리빌드·배열 크기·버퍼 캐시가 아니라 테이블 액세스 자체 제거(커버링 인덱스) 또는 Full Scan 전환입니다.

■ 한 줄 정리 테이블 자체 cr 190,000이 ROWID 200,000과 거의 같아 CF가 최악이고, 이 190,000블록의 랜덤 액세스가 cr의 99.3%·24.5초를 차지하며 손익분기점을 넘었으므로 처방은 인덱스·캐시가 아니라 커버링 인덱스로 테이블 액세스를 없애는 것입니다.

문제 8. 정답 ④

B-Tree 스캔 비용은 성격이 다른 두 축으로 나뉩니다. 이 둘을 섞으면 안 됩니다.

수직(내려가기) : 루트 → 브랜치 → 시작 리프

비용 결정 = 인덱스 높이(BLEVEL+1) ← 보통 2~4로 작고 완만

수평(훑기) : 시작 리프 → 이웃 리프 → ... (Range Scan)

비용 결정 = 스캔 범위(조건 선택도) + 엔트리 밀도/클러스터링

→ 조건이 넓게 걸릴수록 훑는 리프 블록 수가 늘어난다

④는 이 두 축의 경계를 뚫겠습니다. 인덱스 높이는 시작 리프에 닿기까지의 수직 비용을 좌우할 뿐, Range Scan이 수평으로 훑는 리프 블록 수는 스캔 범위(조건이 얼마나 넓은가)로 정해집니다. 높이가 낮아도 조건 범위가 넓으면 많은 리프를 훑고, 높이가 높아도 범위가 좁으면 몇 개만 훑습니다. 그래서 "인덱스 높이가 Range Scan의 리프 스캔량을 결정한다"는 것은 수직 비용의 성질을 수평 비용에 잘못 갖다 붙인 서술입니다.

①·②·③은 옳습니다. 수직 탐색과 높이(①), 리프의 양방향 연결과 수평 이어 읽기(②), Unique Scan이 한 건에서 멈춰 수평 스캔이 없는 성질(③)이 모두 B-Tree 구조에 부합합니다.

선택지	판정	이유
①	○	B-Tree는 루트→브랜치→리프로 수직 탐색하며, 시작 리프에 닿기까지 인덱스 높이(BLEVEL+1)만큼의 블록을 읽습니다
②	○	리프 블록은 좌우 양방향 포인터로 연결돼 있어, 시작 리프 도달 후 이웃 리프를 수평으로 따라가며 엔트리를 이어 읽습니다
③	○	Index Unique Scan은 유일성이 보장돼 최대 한 건을 찾으면 즉시 멈추므로, 리프를 수평으로 이어 읽는 과정이 없습니다
④	×	인덱스 높이는 시작 리프에 닿는 수직 비용을 좌우할 뿐입니다. Range Scan이 훑는 리프 블록 수는 스캔 범위(조건 선택도)로 정해지므로, 높이를 수평 스캔량의 주된 요인으로 본 것은 두 축의 경계를 뚫은 서술입니다

■ 이 문제의 핵심

1. 수직(내려가기)과 수평(훑기)은 다른 축입니다. 수직 비용은 인덱스 높이, 수평 비용은 스캔 범위가 결정합니다.
2. 인덱스 높이(BLEVEL+1)는 시작 리프에 닿기까지 읽는 블록 수(수직)일 뿐, Range Scan이 훑는 리프 블록 수(수평)를 정하지 않습니다.
3. 리프는 좌우 양방향으로 연결돼 있어, Range Scan은 시작 리프부터 이웃 리프를 수평으로 이어 읽습니다.
4. Index Unique Scan = 한 건에서 즉시 종료. 수평으로 이어 읽는 과정 자체가 없습니다.

■ 한 줄 정리 인덱스 높이는 시작 리프에 닿는 수직 비용이고 Range Scan이 훑는 리프 블록 수는 스캔 범위가 결정하므로, 높이가 리프 스캔량을 좌우한다고 보는 것은 수직·수평 두 축의 경계를 뚫은 오해입니다.

문제 9. 정답 ①

핵심은 스캔 범위를 좁히는 조건과 엔트리를 걸러 내는 조건을 가르는 것입니다.

액세스 조건 : 스캔 시작점·종료점을 좁힌다 → 훑는 리프 블록 수 자체를 줄인다

필터 조건 : 스캔 구간은 그대로 두고, 훑으며 각 엔트리를 하나씩 버린다

→ 읽는 리프 블록 수는 줄지 않는다 (버려질 엔트리까지 읽어야 함)

①은 이 경계를 뚫겠습니다. "인덱스에 있는 컬럼 조건이면 액세스든 필터든 스캔 범위를 좁힌다"는 서술은, 필터 조건에 스캔 범위를 좁히는 힘을 잘못 부여한 것입니다. 필터 조건을 아무리 많이 걸어도 인덱스 스캔이 훑는 리프 블록 수는 줄지 않습니다. 걸러지는 건수(결과 행)는 줄지만, 그 판정을 위해 리프 엔트리를 이미 다 읽은 뒤이기 때문입니다. 스캔 범위를 실제로 좁히는 것은 액세스 조건뿐입니다.

②·③·④는 옳습니다. 액세스 조건이 스캔 시작·종료를 정해 리프 블록 수를 직접 줄이고(②), 필터 조건은 구간을 못 줄인 채 버려질 엔트리까지 읽게 하며(③), 결합 인덱스에서 선두 등치부터 첫 범위까지가 액세스 조건이고 그 뒤는 필터로 작동한다(④)는 것이 모두 스캔 효율의 원리에 부합합니다.

선택지	판정	이유
①	×	필터 조건은 스캔 구간을 좁히지 못하므로 아무리 많이 걸어도 읽는 리프 블록 수가 줄지 않습니다. 스캔 범위를 좁히는 것은 액세스 조건뿐인데, 두 조건의 경계를 뚫은 서술입니다
②	○	액세스 조건은 스캔 시작점·종료점을 정해 훑을 리프 구간 자체를 줄이므로, 읽는 리프 블록 수를 직접 감소시킵니다
③	○	필터 조건은 구간 안의 엔트리를 하나씩 평가해 걸러 낼 뿐 구간 크기를 줄이지 못해, 버려질 엔트리가 놓인 리프까지 읽는 비용이 듭니다
④	○	결합 인덱스에서 선두부터 등치로 이어지다 처음 만난 범위까지가 액세스 조건이고, 그 뒤 컬럼 조건은 인덱스에 있어도 필터로만 작동합니다

■ 이 문제의 핵심

1. 액세스 조건만 스캔 범위(읽는 리프 블록 수)를 줄입니다. 필터 조건은 못 줄입니다.
2. 필터 조건은 걸러 낼 뿐입니다. 결과 행은 줄어도, 판정을 위해 리프 엔트리를 이미 다 읽은 뒤라 스캔량은 그대로입니다.
3. 결합 인덱스에서 선두 등치부터 처음 만난 범위까지가 액세스 조건이고, 그 뒤 컬럼 조건은 인덱스에 있어도 필터입니다.

4. 그래서 인덱스 효율은 "인덱스에 컬럼이 있느냐"가 아니라 "그 조건이 스캔 범위를 좁히는 액세스 조건이냐" 로 갑니다.

- 한 줄 정리 읽는 리프 블록 수를 줄이는 것은 스캔 시작·종료를 좁히는 액세스 조건뿐이며, 필터 조건은 아무리 많아도 스캔 범위를 못 줄이므로 "인덱스에 있는 조건이면 필터도 스캔 범위를 좁힌다"는 서술은 두 조건의 경계를 뭉개 오해입니다.

문제 10. 정답 ④

파이프 표에서 HASH JOIN(Id 3)의 두 자식이 어떤 순서로 놓였는지를 읽습니다.

Id 3 HASH JOIN
 Id 4 TABLE ACCESS FULL 단지마스터 (약 5만 건) ← 첫째 자식 = Build Input
 Id 5 TABLE ACCESS FULL 실거래내역 (약 7,500만 건) ← 둘째 자식 = Probe Input

해시 조인은 Build → Probe 두 단계로 움직입니다. 옵티마이저는 두 입력 중 더 작은 쪽을 Build Input으로 잡아 해시 테이블을 Hash Area에 올립니다. 플랜에서 HASH JOIN의 첫째 자식(Id 4) 이 곧 Build Input이고, 여기서는 5만 건짜리 단지마스터입니다. 둘째 자식 Id 5 실거래내역(7,500만 건)이 Probe Input입니다.

Build 단계 : Id 4 단지마스터 5만 건 → Hash Area에 해시 테이블 구축
 Probe 단계 : Id 5 실거래내역 7,500만 건 → 해시 테이블을 조회하며 한 번 훑음

Probe는 NL 조인처럼 반복 탐색하는 것이 아니라, 실거래내역을 한 번 스캔하면서 각 행의 단지코드로 해시 테이블을 조회해 매칭합니다. 큰 테이블(실거래내역)을 딱 한 번만 훑는다는 것이 해시 조인의 이점입니다. ④가 Build/Probe 지목과 조인 순서를 모두 정확히 서술합니다.

- ④ Build=Id 4 단지마스터(작은 쪽), Probe=Id 5 실거래내역(큰 쪽), 순서 Build→Probe
 ① Build/Probe를 뒤집음 (첫째 자식 Id 4가 Build)
 ② 병목을 Build(5만 건)로 오귀속 (실제 비용은 7,500만 Probe Full Scan)
 ③ 해시를 NL의 반복 스캔으로 오해

선택지	판정	이유
①	×	HASH JOIN의 첫째 자식 Id 4(단지마스터, 5만 건)가 Build Input이라 Build/Probe를 뒤집었습니다. 7,500만 건을 Hash Area에 담으면 Temp 분할이 나므로 옵티마이저는 작은 쪽을 Build로 잡습니다. Probe는 해시 조회 한 번이라 "반복 횟수 5만 회"라는 셈 자체가 성립하지 않습니다
②	×	병목을 영동한 입력에 붙였습니다. Id 4 단지마스터 5만 건을 해시 테이블로 올리는 Build는 가볍고, 비용을 좌우하는 것은 Id 5 실거래내역 7,500만 건을 Full Scan하는 Probe 쪽입니다. Id 2·Id 1이 작업 영역을 함께 쓴다는 사실도 Build를 병목으로 만들지 않습니다
③	×	단지마스터 한 행마다 실거래내역을 반복 스캔한다는 것은 NL 조인의 서술입니다. 해시 조인의 Probe는 Id 5를 한 번만 훑으며 해시 테이블을 조회하므로 Full Scan이 5만 번 겹치지 않고, Id 5가 TABLE ACCESS FULL인 것은 조인 인덱스 부재가 아니라 7,500만 건을 통째로 읽는 것이 해시 조인의 정상 경로이기 때문입니다
④	○	Id 4 단지마스터가 Build Input(작은 쪽, 먼저 해시 테이블 구축), Id 5 실거래내역이 Probe Input(한 번 스캔하며 해시 조회), 조인 순서 Build→Probe — 파이프 표의 자식 순서·테이블 크기와 일치합니다

▪ 이 문제의 핵심

1. HASH JOIN의 첫째 자식이 Build Input. 여기서는 Id 4 단지마스터(5만 건)가 Build, Id 5 실거래내역(7,500만 건)이 Probe입니다.
2. 작은 쪽이 Build. 해시 테이블을 Hash Area에 담아 Temp 분할을 피하려는 선택이라, 큰 쪽을 Build로 잡는 진술은 방향이 반대입니다.
3. Probe는 한 번 스캔 + 해시 조회. NL처럼 반복 탐색하지 않으므로, 큰 테이블을 한 번만 훑습니다.
4. 비용은 Probe 쪽 Full Scan에. 5만 건 Build를 병목으로 지목하면 원인을 영동한 입력에 붙이는 오귀속입니다.
 - 한 줄 정리 HASH JOIN의 첫째 자식 Id 4 단지마스터(5만 건)가 Build Input이고 Id 5 실거래내역(7,500만 건)이 Probe Input이라, Build→Probe 순서로 큰 테이블을 한 번만 훑으며 조회하는 ④가 옳고 Build/Probe를 뒤집거나 병목을 Build로 오귀속한 나머지는 틀립니다.

문제 11. 정답 ②

소트 머지 조인은 두 집합을 각각 조인 키로 정렬한 뒤, 정렬된 두 스트림을 앞에서부터 맞대어 병합(merge) 하는 방식입니다.

왼쪽 집합 → 조인 키로 정렬(Sort Join) ↘
 ↳ 두 정렬 스트림을 순서대로 병합(Merge Join)
 오른쪽 집합 → 조인 키로 정렬(Sort Join) ↗

양쪽을 정렬해 두면 매칭을 정렬 순서의 대소 비교로 진행할 수 있으므로, 등치(=)뿐 아니라 <·>·BETWEEN 같은 비등치(범위) 조인도 처리됩니다. 이 점이 등치 매칭만 되는 해시 조인과 갈리는 지점입니다. 그래서 소트 머지 조인은 조인

조건이 비등치이거나, 정렬 부담이 크지 않거나 인덱스·선행 오퍼레이션 덕에 이미 정렬된 입력을 얻을 수 있을 때 해시 조인의 유효한 대안이 됩니다. ②가 처리 방식과 선택 기준을 모두 정확히 서술합니다.

나머지는 '정렬'이라는 키워드에서 곧장 결론으로 건너뛴 반사적 연상입니다.

- ①: '병합'이라는 말에서 "한쪽만 정렬"을 반사적으로 떠올렸지만, 소트 머지 조인은 양쪽을 모두 정렬합니다. 정렬되지 않은 쪽이 크면 그쪽 Sort Join이 오히려 비용의 중심이 되므로, "추가 정렬 없이 병합"이라는 귀결도 성립하지 않습니다.
- ③: '정렬=느림'이라는 반사입니다. 스펴 위험 비교는 그 자체로는 맞지만, 이미 정렬된 입력을 얻거나 비등치 조인처럼 해시가 못 쓰이는 상황에서는 오히려 유리하므로 "느릴 수밖에 없다"·"소량 전용"이라는 분류로 이어지지 않습니다.
- ④: '정렬 생략=NL'이라는 반사입니다. 인덱스로 정렬을 생략해도 두 스트림을 병합하는 구조 자체는 그대로이며, 드라이빙 행마다 인너를 반복 탐색하는 NL과는 다른 방식입니다.

선택지	판정	이유
①	×	소트 머지 조인은 양쪽 집합을 각각 정렬합니다. '병합'이라는 이름에서 한쪽만 정렬한다고 반사적으로 연결했고, 그 탓에 "정렬 안 된 쪽이 커도 추가 정렬 없이 병합"이라는 결론까지 따라 나왔습니다. 실제로는 그쪽 Sort Join이 비용의 중심이 됩니다
②	○	양쪽을 조인 키로 정렬해 병합하므로 비등치·범위 조인도 처리하며, 정렬 부담이 작거나 이미 정렬된 입력이 있을 때 해시 조인의 대안이 됩니다
③	×	해시 조인이 작은 쪽만 올려 스펴 위험이 낮다는 대목은 맞지만, 거기서 "대량에서 느릴 수밖에 없다·소량 전용"이라는 분류가 나오지 않습니다. 이미 정렬된 입력을 얻거나 해시가 못 쓰이는 비등치 조인에서는 오히려 유리합니다
④	×	정렬을 생략해도 두 정렬 스트림을 병합하는 구조는 그대로입니다. 드라이빙 행마다 인너 인덱스를 반복 탐색하는 것은 NL 조인이고, 병합은 양쪽 스트림을 한 번씩 앞으로 밀며 대소를 비교할 뿐이라 조인 순서 민감도도 NL과 다릅니다

■ 이 문제의 핵심

1. 양쪽을 각각 정렬한 뒤 병합하는 것이 소트 머지 조인입니다. 한쪽만 정렬하는 방식이 아닙니다.
2. 비등치(범위) 조인을 처리할 수 있습니다. 정렬 순서의 대소 비교로 병합하므로 등치만 되는 해시 조인과 같습니다.
3. 선택 기준은 정렬 부담과 입력의 정렬 여부입니다. 이미 정렬된 입력을 얻거나 비등치 조인이면 해시 조인의 대안이 됩니다.
4. 정렬을 생략해도 병합 방식은 그대로입니다. NL처럼 드라이빙 건마다 인너를 반복 탐색하는 구조로 바뀌지 않으며, 어느 쪽을 먼저 읽느냐(조인 순서)에 NL만큼 민감하지 않습니다.

■ 한 줄 정리 소트 머지 조인은 양쪽을 각각 조인 키로 정렬해 병합하므로 비등치 조인도 처리하고 정렬 부담이 작거나 이미 정렬된 입력이 있을 때 해시 조인의 대안이 되며, "한쪽만 정렬"·"정렬=무조건 느림"·"정렬 생략=NL"은 키워드에서 곧장 결론으로 건너뛴 반사적 연상입니다.

문제 12. 정답 ③

GROUP BY나 **DISTINCT**가 든 복합 뷰도 옵티마이저가 병합할 수 있습니다. 이것을 복합 뷰 Merging이라 부릅니다.

복합 뷰 = 집계/중복제거가 든 뷰

병합 가능 → 집계(GROUP BY·DISTINCT)의 수행 시점을 조인 앞뒤로 재배치

- 뷰에서 먼저 집계한 뒤 조인
- 조인을 먼저 하고 집계를 뒤로 미룸(delayed grouping)

둘 중 비용이 낮은 형태를 옵티마이저가 선택

③은 "집계 뷰는 병합 대상에서 제외되어 개별 수행된 뒤에만 조인된다"고 단정합니다. 이는 "집계가 있으니 병합은 안 된다"는 반사적 연상으로, 실제 옵티마이저 동작과 어긋납니다. 옵티마이저는 복합 뷰를 병합해 집계 시점을 조인 뒤로 미루는(또는 앞당기는) 재구성을 할 수 있으므로, "수행 순서를 바꾸지 못한다"는 진술이 틀렸습니다.

나머지는 모두 옳습니다.

- ①: 단순 뷰 병합은 정의를 그대로 풀어 넣지만, 복합 뷰 병합은 집계·중복 제거 연산을 함께 재배치해야 하므로 처리 방식이 다릅니다.
- ②: 복합 뷰 병합은 집계를 조인 뒤로 미루는 형태와 먼저 집계하는 형태를 비용으로 비교해 고를 수 있습니다.
- ④: 복합 뷰 병합이 늘 이득은 아니므로 비용으로 판단하며, **NO_MERGE/MERGE** 힌트로 억제·유도할 수 있습니다.

선택지	판정	이유
①	○	단순 병합은 정의를 그대로 풀어 넣고, 복합 뷰 병합은 집계·중복 제거까지 재배치해야 하므로 처리 방식이 다릅니다
②	○	복합 뷰 병합은 집계를 조인 뒤로 미루거나 먼저 집계하는 두 형태를 비용으로 비교해 고릅니다
③	×	집계 뷰도 복합 뷰 Merging으로 병합돼 집계 시점이 조인 앞뒤로 재배치될 수 있습니다. "집계가 있으니 병합 불가"는 반사적 연상이며, 수행 순서를 못 바꾼다는 단정이 틀렸습니다
④	○	복합 뷰 병합은 늘 이득이 아니라 비용으로 판단하며, NO_MERGE/MERGE 힌트로 억제·유도됩니다

▪ 이 문제의 핵심

1. **GROUP BY·DISTINCT** 복합 뷰도 병합될 수 있습니다. 이것이 복합 뷰 Merging입니다.
2. 집계 시점의 재배치가 핵심입니다. 조인을 먼저 하고 집계를 뒤로 미루거나, 먼저 집계한 뒤 조인하는 형태 중 비용으로 고릅니다.
3. 단순 병합과 처리 방식이 다릅니다. 정의를 그대로 풀어 넣는 단순 병합과 달리 집계·중복 제거 연산을 함께 옮겨야 합니다.
4. 비용 기반이며 힌트로 제어합니다. **NO_MERGE**로 막고 **MERGE**로 유도합니다 — "집계 뷰는 병합 불가"는 반사적 연상입니다.

▪ 한 줄 정리 **GROUP BY·DISTINCT** 복합 뷰도 복합 뷰 Merging으로 병합돼 집계 시점이 조인 앞뒤로 재배치될 수 있으므로, "집계 뷰는 병합 대상에서 제외되어 수행 순서를 바꾸지 못한다"는 진술은 집계라는 키워드에서 곧장 '병합 불가'로 건너뛰 반사적 연상입니다.

문제 13. 정답 ①

동적 SQL이라도 커서 공유의 기준은 SQL 텍스트가 문자 단위로 같은가입니다. 방식 (가)는 :g 플레이스홀더로 텍스트가 고정되어 PARSE 한 번에 커서 하나이고, 방식 (나)는 차량구분 값을 리터럴로 이어 붙이므로 코드가 바뀔 때마다 텍스트가 달라져 매번 새 커서로 하드파싱됩니다.

①은 "리터럴만 달라도 기본 모드가 알아서 바인드처럼 묶어 준다"는 정규화 가정의 오류입니다. 기본 커서 공유 모드(**CURSOR_SHARING=EXACT**)에서 두 SQL이 부모 커서를 공유하려면 텍스트가 문자 단위로 완전히 같아야 합니다 — 공백·대소문자·주석 하나만 달라도 서로 다른 SQL_ID로 갈립니다. 방식 (나)는 'CAR'·'BUS'처럼 리터럴 값을 직접 이어 붙이므로 코드가 바뀔 때마다 텍스트 자체가 달라져 매번 하드파싱됩니다. EXACT 모드는 리터럴을 바인드로 자동 치환하지 않습니다(그 동작은 **CURSOR_SHARING=FORCE**에서만 일어납니다). 리터럴을 값과 무관하게 하나로 묶으려면 방식 (가)처럼 바인드 플레이스홀더를 쓰거나 **USING** 절로 바인딩해야 합니다. 그래서 부적절한 진술은 ①입니다.

②③④는 각각 DBMS_SQL의 1회 파싱·반복 실행(②), 리터럴 결합의 반복 하드파싱과 USING 바인딩의 텍스트 고정(③), 바인드 변수의 커서 공유·주입 완화(④)를 옳게 서술합니다.

선택지	판정	이유
①	×	기본(EXACT) 모드는 텍스트가 문자 단위로 같아야 커서를 공유하며, 리터럴을 바인드처럼 자동 치환하는 것은 FORCE 모드뿐입니다. (나)는 값마다 텍스트가 달라 코드 종류만큼 커서가 갈립니다
②	○	방식 (가)는 PARSE를 루프 밖 1회만 하고 BIND로 값만 바꿔 EXECUTE를 반복하므로 파싱된 커서를 재사용합니다
③	○	방식 (나)는 리터럴 결합이라 값 종류만큼 텍스트가 달라져 하드파싱이 반복되고, USING 바인딩이면 텍스트가 :b 하나로 고정돼 공유됩니다
④	○	DBMS_SQL·NDS 모두 바인드 변수를 쓰면 텍스트가 고정되어 커서를 공유하고, 입력을 구문으로 해석하지 않아 주입 위험도 줄입니다

▪ 이 문제의 핵심

1. 동적 SQL도 커서 공유 기준은 텍스트 일치입니다. 리터럴 결합은 값마다 텍스트가 달라져 하드파싱이 반복됩니다.
2. 기본(EXACT) 공유 기준은 텍스트 완전 일치입니다. 공백·대소문자·주석·리터럴 어느 하나만 달라도 다른 커서이며, 리터럴 자동 바인딩은 FORCE 모드에서만 일어납니다.
3. DBMS_SQL은 PARSE 1회 + BIND/EXECUTE 반복으로 커서를 재사용하고, NDS는 **USING**으로 같은 효과를 냅니다.
4. 바인드 변수는 텍스트를 고정해 커서 공유와 SQL Injection 완화를 함께 얻습니다.

▪ 한 줄 정리 리터럴 결합 동적 SQL은 값마다 텍스트가 달라져 반복 하드파싱되며, 기본(EXACT) 모드는 텍스트 완전 일치만 공유하고 리터럴을 자동 치환하지 않으므로(FORCE에서만) "구조가 같으니 알아서 하나로 공유된다"는 것은 착각입니다.

문제 14. 정답 ②

여기에는 서로 독립인 두 축이 있습니다.

- 축 A. 최적화 목표 : 전체범위(ALL_ROWS, 총 처리량) vs 부분범위(FIRST_ROWS, 응답 시간)
 → 무엇을 최소화할지, 즉 '목표'를 정하는 축 (파라미터·힌트로 지정)
- 축 B. 통계 정확도 : 행 수·NDV·히스토그램 등 통계가 실제 분포와 얼마나 맞는지
 → 어느 목표든 비용을 '얼마나 정확히' 계산하느냐를 좌우하는 축

②는 이 두 축을 뒤섞지 않고 바르게 진술합니다. 전체범위 최적화는 결과 전체를 끝까지 반환하는 총비용을, 부분범위 최적화는 앞쪽 일부를 빨리 내보내는 응답 시간을 최소화하는 서로 다른 목표입니다. 그리고 어느 목표를 택하든 후보 계획의 비용은 통계정보로 산정합니다 — 목표(축 A)와 통계(축 B)는 별개이며, 통계는 두 목표 양쪽에 공통으로 작용합니다.

나머지는 이 독립인 두 축을 인과로 엮은 별개 축 혼동입니다.

①: 최적화 목표는 통계 정확도가 아니라 파라미터(OPTIMIZER_MODE)나 힌트로 정합니다. 행 수·NDV·히스토그램이 정확해지면 카디널리티 추정이 좋아질 뿐, 목표 자체가 자동으로 바뀌지는 않습니다.

③: 통계 정확도는 두 목표 양쪽의 비용 산정에 작용합니다. 전체범위 계획도 카디널리티 오차를 그대로 물려받아 조인 방식·순서를 잘못 고르므로, 통계가 낡았다고 ALL_ROWS가 안전해지지 않습니다.

④: 통계 최신화는 비용 추정을 정확하게 할 뿐, 그 자체가 부분범위 최적화를 켜는 스위치가 아닙니다. 목표 전환은 모드·힌트로 이뤄집니다.

선택지	판정	이유
①	×	최적화 목표는 통계 정확도가 정하는 것이 아니라 OPTIMIZER_MODE·힌트로 지정합니다. 행 수·NDV·히스토그램이 정밀해지면 카디널리티 추정이 좋아질 뿐이며, 목표(축 A)와 통계(축 B)를 인과로 엮은 오류입니다
②	○	전체범위는 총비용, 부분범위는 응답 시간을 최소화하는 서로 다른 목표이며, 어느 목표든 후보 계획 비용은 통계로 산정됩니다
③	×	통계 정확도는 두 목표 양쪽의 비용 산정에 작용합니다. 전체범위 계획도 카디널리티 오차를 물려받아 조인 방식·순서를 잘못 고르므로, 통계가 낡았을 때 ALL_ROWS가 안전해진다는 귀결은 별개 축을 인과로 엮은 혼동입니다
④	×	통계 최신화는 비용 추정을 정확하게 할 뿐 목표를 바꾸지 않습니다. 실제 행 수를 알아도 옵티마이저가 전체범위를 부분범위로 갈아타지는 않으며, 부분범위 최적화는 모드·힌트로 지정합니다

▪ 이 문제의 핵심

1. 최적화 목표와 통계 정확도는 독립인 두 축입니다. 목표는 '무엇을 최소화할지', 통계는 '비용을 얼마나 정확히 계산할지'를 정합니다.
2. 전체범위 = 총비용, 부분범위 = 응답 시간. 서로 다른 최소화 대상입니다.
3. 통계는 두 목표 양쪽에 공통으로 작용합니다. 어느 목표를 택하든 비용은 통계정보로 산정됩니다.
4. 목표는 모드·힌트로 지정합니다. 통계가 정확·최신이라고 목표가 자동으로 바뀌거나 켜지지 않습니다.

▪ 한 줄 정리 전체범위(총비용)와 부분범위(응답 시간)는 서로 다른 최적화 목표이고 통계정보는 그 목표와 무관하게 두 경우 양쪽의 비용 산정에 작용하는 별개 축이므로, "통계 정확도가 목표를 정한다"거나 "목표에 따라 통계 민감도가 갈린다"는 진술은 독립인 두 축을 인과로 엮은 오류입니다.

문제 15. 정답 ④

대량 INSERT의 DML 부하는 데이터 블록 변경만이 아니라 그에 딸린 인덱스 유지·제약 검증·트리거 실행에서 옵니다. 인덱스가 많을수록 행마다 갱신할 인덱스 블록이 늘어 redo·undo가 함께 커집니다(①③이 옳게 서술). 그래서 적재 전 인덱스를 UNUSABLE로 돌려 유지 비용을 끊고, 적재 후 한꺼번에 REBUILD하는 전략이 유효합니다(②).

④는 Direct Path Insert의 동작을 정반대로 뒤집었습니다. Direct Path Insert(APPEND)는 버퍼 캐시를 우회해 HWM 위에 블록을 직접 붙이며, 이 경로에서 테이블 데이터에 대한 언두를 거의 남기지 않습니다(그래서 일반 경로보다 undo가 적고, NOLOGGING과 결합하면 redo까지 최소화됩니다). 인덱스도 행마다 실시간으로 하나씩 갱신하는 것이 아니라, 적재가 끝난 뒤 정렬해 일괄(bulk) 병합하는 편이라 오히려 인덱스 유지 부담이 완화된다는 것인데 ④는 "언두를 크게 남기고 인덱스를 실시간 갱신하므로 항상 더 크다"고 했으니, 언두·인덱스 유지 방향을 모두 반대로 서술한 것입니다. 그래서 부적절한 진술은 ④입니다.

①②③은 각각 인덱스·제약·트리거로 인한 DML 부하 증가(①), UNUSABLE→REBUILD의 유지 비용·redo 절감(②), 인덱스 개수와 redo·undo 증가의 비례(③)를 옳게 서술합니다.

선택지	판정	이유
①	○	인덱스 4개·PK·FK·트리거가 있으면 행마다 엔트리 추가·제약 검증·트리거 실행이 없혀 순수 적재보다 DML 부하가 큼니다
②	○	인덱스를 UNUSABLE로 두고 적재 후 REBUILD하면 적재 중 인덱스 유지 비용과 인덱스 redo가 줄어 시간이 단축됩니다
③	○	INSERT는 테이블과 인덱스 블록을 함께 변경하므로 redo·undo가 늘고, 인덱스가 많을수록 그 양이 증가합니다
④	×	Direct Path Insert는 언두를 거의 안 남기고 인덱스도 일괄 병합해 부담을 완화합니다. "언두를 크게 남기고 실시간 갱신해 항상 더 크다"는 방향이 반대입니다

▪ 이 문제의 핵심

1. 대량 INSERT의 DML 부하는 데이터 변경 + 인덱스 유지 + 제약 검증 + 트리거의 합입니다. 인덱스가 많을수록 커집니다.
2. redo·undo는 테이블과 인덱스 블록 변경에서 함께 발생하므로, 인덱스 개수에 비례해 증가합니다.
3. 적재 전 UNUSABLE → 적재 후 REBUILD는 적재 중 인덱스 유지 비용과 관련 redo를 끊어 시간을 줄입니다.
4. Direct Path Insert는 언두를 거의 안 남기고 인덱스를 일괄 병합해 부하를 낮추는 경로입니다 — 실시간 갱신으로 더 무거워진다는 것은 반대입니다.

▪ 한 줄 정리 대량 INSERT의 부하는 인덱스·제약·트리거 유지에서 오고 인덱스를 UNUSABLE→REBUILD로 끊어 줄일 수 있는데, Direct Path Insert는 언두·인덱스 유지를 오히려 완화하는 경로이므로 "언두를 크게 남기고 실시간 갱신해 항상 더 크다"는 ④는 방향을 뒤집은 진술입니다.

문제 16. 정답 ③

로컬 인덱스는 데이터 파티션과 1:1로 짝을 이루지만, 불필요한 인덱스 파티션을 건너뛰는 프루닝이 되려면 파티션 키 조건이 있어야 하고, 그 판정은 인덱스가 prefixed냐 nonprefixed냐로 갈립니다.

[로컬 인덱스별 파티션 접근 범위]

ix_loc_pfx (이용일시, 노선번호) → 선두가 파티션 키
WHERE 이용일시 >= '2025-08-01' → 프루닝 → 대응 인덱스 파티션만
ix_loc_npfx (카드번호, 이용일시) → 선두가 파티션 키 아님
WHERE 카드번호 = :1 → 어느 파티션인지 모름 → 전 인덱스 파티션 프로빙
WHERE 카드번호 = :1 AND 이용일시 >= ... → 그제야 프루닝 가능

③은 "로컬 인덱스이다"라는 부분적 사실만 딴고, nonprefixed 인덱스가 카드번호 단독 조회에서도 "프루닝이 걸려 단일 파티션만 보고, 효율이 프리픽스와 동일하다"고 단정합니다. 그러나 로컬이라는 성질만으로 프루닝이 보장되지는 않습니다. ix_loc_npfx는 선두가 카드번호라, 이용일시 조건이 없으면 특정 카드의 이력이 어느 월 파티션에 있는지 알 수 없어 모든 인덱스 파티션을 훑어야 합니다(②가 옳게 서술). 파티션 키 조건이 함께 주어져야 비로소 프리픽스처럼 소수 파티션으로 좁혀집니다(④). 즉 "로컬"이라는 한 조건이 최선(1개 파티션)을 최악(전 파티션 프로빙)으로 뒤집는 지점을 무시한 부분진실 진술이 ③이고, 이는 ②와 정면으로 모순됩니다. 그래서 부적절한 것은 ③입니다.

①②④는 프리픽스 로컬의 프루닝(①), nonprefixed의 전 파티션 프로빙(②), 파티션 키 조건 결합 시의 프루닝 회복(④)을 모두 옳게 서술합니다.

선택지	판정	이유
①	○	ix_loc_pfx는 선두가 파티션 키(이용일시)라 날짜 조건이 오면 프루닝으로 대응 인덱스 파티션만 스캔합니다
②	○	ix_loc_npfx는 선두가 카드번호라, 이용일시 조건이 없으면 어느 파티션인지 몰라 전 인덱스 파티션을 프로빙합니다
③	×	nonprefixed 로컬은 파티션 키 조건이 없으면 전 인덱스 파티션을 훑으므로 프루닝 효과가 프리픽스와 같지 않습니다. "로컬이니 단일 파티션"은 부분진실입니다
④	○	ix_loc_npfx라도 이용일시 조건을 함께 주면 그제야 프루닝이 적용돼 접근 인덱스 파티션 수가 줄어듭니다

▪ 이 문제의 핵심

1. 로컬 인덱스라는 사실만으로 파티션 프루닝이 보장되지 않습니다. 관건은 파티션 키 조건과 prefixed/nonprefixed 여부입니다.
2. 로컬 프리픽스(이용일시, ...)는 파티션 키 조건이 오면 대응 인덱스 파티션만 스캔합니다.
3. 로컬 네프리픽스(카드번호, ...)는 파티션 키 조건이 없으면 모든 인덱스 파티션을 프로빙합니다 — 이때 프리픽스와 효율이 다릅니다.
4. 파티션 키 조건을 결합하면 네프리픽스도 프루닝이 회복되어 접근 파티션 수가 줄어듭니다.

ROWS BETWEEN ... : 물리적 '행 개수'로 프레임을 셈
 → 동순위가 있어도 각 행이 별개로 세어짐
 RANGE BETWEEN ... : ORDER BY '값'으로 프레임을 셈
 → 값이 같은 동순위 행은 하나의 논리적 범위로 묶임

그래서 **ORDER BY** 기준 값에 동순위(ties)가 있을 때, 같은 입력·같은 **ORDER BY**라도 누적 합(running total) 같은 결과가 두 프레임에서 달라질 수 있습니다. 예컨대 같은 날짜가 여러 건이면 **RANGE**는 그 날짜의 전체를 한꺼번에 누적하고, **ROWS**는 현재 행까지만 누적합니다. ③이 두 프레임의 차이와 그 결과 영향을 정확히 서술합니다.

나머지는 일부 경우에만 성립하는 성질을 전체로 부풀린 하위항목 전체화 오류입니다.

- ①: 분석함수들이 같은 정렬 기준을 공유할 때만 하나의 Window Sort로 처리됩니다. **ORDER BY** 기준이 서로 다르면 기준마다 별도의 Window Sort가 필요하므로, "정렬 한 번이면 충분"은 한 갈래를 전체로 부풀린 것입니다.
- ②: **UNION ALL** 자체는 Sort Unique를 안 하는 것이 맞지만, 바깥에 **ORDER BY**를 붙이면 그 전체 정렬을 위한 SORT ORDER BY 오퍼레이션이 별도로 추가됩니다. "정렬이 붙지 않는다"는 것은 연산 자체의 성질을 결과 전체로 일반화한 오류입니다.
- ④: 집계형 분석함수라도 **SUM(...)** **OVER(ORDER BY ...)** 같은 누적 계산은 **ORDER BY** 순서로 정렬(Window Sort)해야 합니다. 순위 함수만 정렬이 필요하다는 것은 정렬이 필요한 경우를 순위 함수로 좁혀 버린 오류입니다.

선택지	판정	이유
①	×	하나의 Window Sort로 묶이는 것은 분석함수들이 같은 정렬 기준을 공유할 때뿐입니다. ORDER BY 기준이 다르면 기준마다 별도 정렬이 필요합니다
②	×	UNION ALL 자체는 Sort Unique가 없지만, 바깥 ORDER BY 를 붙이면 전체 정렬을 위한 SORT ORDER BY가 별도로 추가됩니다
③	○	ROWS 는 물리적 행 수, RANGE 는 ORDER BY 값으로 프레임을 세므로, 동순위가 있으면 같은 입력이라도 두 프레임의 누적 결과가 달라질 수 있습니다
④	×	SUM(...) OVER(ORDER BY ...) 같은 누적 집계도 ORDER BY 순서로 Window Sort가 필요합니다. 정렬이 필요한 경우를 순위 함수로 좁힌 오류입니다

▪ 이 문제의 핵심

1. **ROWS**는 행 개수, **RANGE**는 **ORDER BY** 값으로 프레임을 셈니다. 동순위가 있으면 두 프레임의 누적 결과가 같립니다.
2. 같은 정렬 기준을 공유할 때만 Window Sort 하나로 처리됩니다. 기준이 다르면 기준마다 별도 정렬이 생깁니다.
3. **UNION ALL**은 Sort Unique가 없지만, 바깥 **ORDER BY**는 별도 정렬을 추가합니다. 연산 자체의 성질을 결과 전체로 일반화하면 안 됩니다.
4. 누적 집계도 정렬이 필요합니다. **SUM(...)** **OVER(ORDER BY ...)**는 순위 함수가 아니어도 Window Sort를 씁니다.

▪ 한 줄 정리 **ROWS**와 **RANGE**는 프레임에 세는 단위가 달라 동순위가 있으면 누적 결과가 같리며, "정렬 기준이 달라도 한 번이면 충분"·"**UNION ALL**엔 정렬이 붙지 않음"·"집계형은 **ORDER BY**를 써도 정렬 불필요"는 일부 성질을 전체로 부풀린 하위항목 전체화 오류입니다.

문제 19. 정답 ①

READ UNCOMMITTED는 다른 트랜잭션이 아직 커밋하지 않은 변경분까지 읽습니다. 그래서 TX1이 감소만 하고 커밋하지 않은 상태에서 TX2가 그 값을 읽고, 이후 TX1이 롤백하면 TX2는 실재하지 않았던 값을 읽은 셈이 됩니다 — 이것이 Dirty Read입니다.

[타임라인 - 잔여좌석(S7) 초기 20]

t1 TX1 UPDATE = 20 - 6 = 14 (미커밋)
 t2 TX2(RU) SELECT → (가) = 14 (커밋 안 된 값을 읽음)
 t3 TX1 ROLLBACK → 20 복원 (14는 확정된 적 없는 값 = Dirty Read)

[격리수준별 t2의 (가)]

READ UNCOMMITTED : (가) = 14 → Dirty Read 발생
 READ COMMITTED : (가) = 20 → 미커밋 14가 안 보임 → Dirty Read 없음

READ UNCOMMITTED에서 (가)=14이고, TX1이 롤백해 그 값은 확정된 적이 없으므로 Dirty Read가 맞습니다. READ COMMITTED 이상이면 커밋된 값만 읽어 t2는 20을 보고 현상이 사라집니다. ①이 값·현상·격리수준 상향 효과를 모두 맞혔습니다.

②는 RU가 공유 락을 걸지 않는다는 참인 사실에서 "커밋된 값만 읽는다"로 건너뛰었고(실제로는 미커밋 14를 읽음), ③은 초기값 20과 t2의 단 한 번의 읽기를 재조회 2회로 오분류했으며, ④는 "락 대기 없이 읽음"이라는 RU·RC에 공통인 단서를 판별 근거로 삼은 오류입니다.

선택지	판정	이유
①	○	RU라 t2에서 미커밋 14를 읽어 (가)=14, t3 롤백으로 확정된 적 없는 값을 읽은 Dirty Read이며, RC 이상이면 20을 봐 사라집니다
②	×	공유 락을 걸지 않는다는 대목은 맞지만, RU는 거기서 더 나아가 미커밋 변경분까지 읽습니다. t2가 보는 값은 t3에서 복원될 20이 아니라 TX1이 만든 14이고, 그래서 Dirty Read가 발생합니다
③	×	TX2는 t2에서 한 번만 읽었습니다. 20과 14가 관측됐다는 것은 초기값과 t2 읽기를 나란히 놓은 것일 뿐 TX2의 재조회가 아니므로, Non-Repeatable Read가 아니라 Dirty Read이고 RC만 되어도 사라집니다
④	×	"락 대기 없이 읽음"은 RU·RC에 공통이라 현상 판별 근거가 못 됩니다. RC였다면 대기 없이 읽되 값은 20이었을 것이므로, 대기 유무가 아니라 t1의 미커밋 14를 읽었는지가 Dirty Read를 가릅니다

▪ 이 문제의 핵심

1. Dirty Read = 커밋되지 않은 변경분을 읽고, 그 트랜잭션이 롤백되어 실재하지 않았던 값을 읽은 현상입니다. 여기서 (가)=14가 그 값입니다.
2. READ UNCOMMITTED는 미커밋 변경을 읽습니다. READ COMMITTED 이상은 커밋된 값만 읽어 Dirty Read를 막습니다.
3. Dirty Read와 Non-Repeatable Read의 구분: Dirty는 미커밋 값을 읽는 것(1회 읽기로도 성립), NRR은 재조회 사이 커밋된 변경으로 값이 달라지는 것(2회 읽기).
4. "락 대기 없이 즉시 읽었다"는 RU·RC 공통 단서라 이상현상의 판별 근거가 아닙니다.

▪ 한 줄 정리 READ UNCOMMITTED는 미커밋 값을 읽으므로 (가)=14이고 TX1의 롤백으로 Dirty Read가 되며, "락 대기 없이 읽었다"는 공통 사실만으로는 현상을 가릴 수 없습니다.

문제 20. 정답 ③

SELECT ... FOR UPDATE는 조회한 행에 TX 락을 배타(X, 6) 모드로 잡습니다. 표에서 세션 27이 **LMODE=6, BLOCK=1**로 7180 행을 배타 점유하고, 세션 38이 같은 행에 **REQUEST=6**으로 대기하는 그림이 바로 그 충돌입니다. 같은 행을 배타로 원하는 두 세션은 공존할 수 없으므로, 뒤에 온 세션은 앞 세션이 커밋/롤백으로 락을 놓을 때까지 기다립니다(②).

③은 **NOWAIT**의 동작을 정반대로 뒤집었습니다. **FOR UPDATE NOWAIT**는 대상 행이 이미 잠겨 있으면 대기하지 않고 즉시 오류(ORA-00054, resource busy)를 던지며 락을 얻지 못합니다. "대기 없이 즉시 획득해 갱신을 이어간다"가 아니라 "대기 없이 즉시 실패한다"가 옳습니다. 표의 세션 27이 7180 행을 X(6)로 점유 중이므로, 세션 38이 **NOWAIT**로 시도하면 REQUEST 상태로 매달리지도 못하고 곧바로 오류로 튕깁니다. 그래서 표가 보여 주는 잠금 상태와 정면으로 어긋나는 진술이 ③이고, 부적절한 것은 ③입니다.

①②④는 각각 FOR UPDATE가 건 행 단위 TX 배타 락(①), 기본 WAIT의 무한 대기(②), **SKIP LOCKED**가 잠긴 행을 건너뛰고 나머지만 잠그는 동작(④)을 옳게 설명합니다.

선택지	판정	이유
①	○	세션 27은 FOR UPDATE로 7180 행에 TX X(6)를 잡고 BLOCK=1로 상대를 막습니다. 테이블 전체가 아니라 그 행에 대한 잠금입니다
②	○	기본 FOR UPDATE(WAIT)는 X(6)를 REQUEST한 채, 세션 27이 커밋·롤백으로 락을 놓을 때까지 대기합니다. 표의 두 번째 행 그대로입니다
③	×	FOR UPDATE NOWAIT 는 이미 잠긴 행이면 즉시 ORA-00054로 실패하지 락을 획득하지 않습니다. "대기 없이 즉시 획득"은 방향이 반대입니다
④	○	SKIP LOCKED 는 잠긴 7180 행을 건너뛰고 잠기지 않은 다른 행만 잠가 대기 없이 처리 대상을 확보합니다

▪ 이 문제의 핵심

1. **SELECT ... FOR UPDATE**는 조회 행에 TX 락을 X(6) 모드로 잡습니다. 테이블 전체가 아니라 행 단위 잠금입니다.
2. 기본(WAIT)은 상대가 커밋·롤백할 때까지 대기합니다. 표의 REQUEST=6·BLOCK=1이 그 충돌을 보여 줍니다.
3. **NOWAIT**는 이미 잠긴 행이면 즉시 ORA-00054로 실패하고 락을 얻지 못합니다 — 즉시 획득이 아닙니다.
4. **SKIP LOCKED**는 잠긴 행을 건너뛰고 나머지만 잠가 대기 없이 처리 대상을 확보합니다(큐 소비 패턴).

▪ 한 줄 정리 **SELECT ... FOR UPDATE**가 건 행 단위 TX X(6) 락은 같은 행을 원하는 세션을 대기시키며, **NOWAIT**는 이미 잠긴 행에서 즉시 실패하므로 "대기 없이 즉시 획득한다"는 ③은 옵션 동작을 정반대로 뒤집은 진술입니다.